claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-a24f5a83838336f4f.jsonl`
- SHA-256: `06ce72546bb7d77fcf2036fc983ba2f14e0adb75f40a1a839287b0ffd8e3c358`
- Source modified: `2026-06-20T09:02:05+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`

Transcript

1. user (2026-06-20T08:57:11.749Z)

Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off origin/master. The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the worker.
Files changed/added (read them in the WORKING TREE ? changes are uncommitted):
- backend/pipeline/detectors/native_wasb_runner.py (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py (added --device, threaded device into _build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py (_resolve_runner: new 'native' branch + _build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py (_build_native_runner override; WASB substrate only)
- backend/config/models.py (WasbIndexConfig.device + .python_bin; detector_impl doc mentions 'native')
- backend/worker/__main__.py (cmd_train: --trajectory-source {synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)

DESIGN INTENT / CONTRACTS (judge against these):
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner, WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively (no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename '{stem}__{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) -> identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-platform path handling matters.
- weights/repo/python/scratch come from env first (WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.

YOU ARE THE NO-WSL / CROSS-PLATFORM SKEPTIC. Confirm native_wasb_runner.py carries NO WSL/conda-activate/mnt plumbing (no WslCommandMixin, no 'wsl', no 'conda activate', no to_wsl_mnt_path) and find any leak. Then hunt cross-platform bugs: the code runs on LINUX but tests run on WINDOWS ? check os.path.expanduser, os.path.abspath, backslash normalization for the key, os.path.join for cwd/'src', shutil.copy/rmtree, subprocess list-form (no shell quoting needed). Will _repo_src()/cwd be correct on Linux? Will run_predict's os.path.abspath(video) mangle a real Linux path? Report concrete bugs.

2. assistant (2026-06-20T08:57:14.194Z)

I'll review this change as the no-WSL/cross-platform skeptic. Let me start by reading the key files.

3. user (2026-06-20T08:57:15.151Z)

```
1       """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3       The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4       "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5       It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6       Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
7       that the WSL adapter runs, so the trajectory CSV is byte-identical ? but WITHOUT the WSL
8       transport:
9
10        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
14          into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
15        * weights / repo / python / scratch come from the **environment** first
16          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
17          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
18
19      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
20      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
21      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
22      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
23      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
24
25      **Acceptance gate (owner / GPU box):** byte-parity ? the SAME clip through the WSL
runner
26      (Windows) and this runner (Linux, ``device=cuda``) yields an identical trajectory CSV
and
27      identical candidate windows. That parity is a hardware check (a real GPU + the ``wasb``
28      env), so it is verified by the owner on the box ? not in this offline, subprocess-mocked
29      test suite.
30      """
31      import os
32      import shutil
33      import logging
34      import subprocess
35      from dataclasses import dataclass
36      from typing import Any, Dict, List, Optional, Tuple
37
38      from backend.pipeline.detectors.base import DetectorRunner
39      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
40      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
41
42      logger = logging.getLogger(__name__)
43
44      # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
45      # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
46      _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
47
48
49      @dataclass
50      class NativeWasbConfig:
51          """Resolved settings for a Linux-native WASB run.
52
```

```
53          Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
54          over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
55          defaults to ``cuda`` (the WSL byte-parity path).
56          """
57          repo_dir: str = "~/models/WASB-SBDT"   # native clone of nttcom/WASB-SBDT
58          python_bin: str = "python"             # the `wasb` conda env python (invoked by
path, no activate)
59          weights_path: str = ""                 # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
60          sport: str = "badminton"
61          device: str = "cuda"                   # cuda | mps | cpu
62          scratch_dir: str = ""                  # frame-extraction scratch (env); "" = next
to the output dir
63          timeout_sec: int = 1800               # 30 min; a hung GPU call is killed (0 = no
timeout)
64          keep_frames: bool = False              # retain the decoded frame cache after
success (debug only)
65          stream_video: bool = False             # fast path: decode in-process, no PNG
extraction (bit-identical)
66
67          @classmethod
68          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
69              w = (idx_cfg or {}).get("wasb", {}) or {}
70              env = os.environ.get
71              return cls(
72                  repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
73                  python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
74                  weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
75                  sport=w.get("sport", "badminton"),
76                  device=env("WASB_DEVICE") or w.get("device", "cuda"),
77                  scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
"",
78                  timeout_sec=int(w.get("timeout_sec", 1800)),
79                  keep_frames=bool(w.get("keep_frames", False)),
80                  stream_video=bool(w.get("stream_video", False)),
81              )
82
83
84      class NativeWasbRunner(DetectorRunner):
85          """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
86
87          def __init__(self, cfg: NativeWasbConfig):
88              self.cfg = cfg
89
90          # --- low-level native exec (the single place tests patch)
--------------------------
91          def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
92              logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
93              return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
94                                    timeout=(self.cfg.timeout_sec or None))
95
96          def _repo_src(self) -> str:
97              return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
98
99          def _copy_infer_script(self) -> bool:
100             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
101             repo_src = self._repo_src()
102             try:
103                 os.makedirs(repo_src, exist_ok=True)
104                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
105                 return True
```

```
106             except OSError as e:
107                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
108                 return False
109 
110         def _rm_rf(self, path: Optional[str]) -> None:
111             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
112             if path:
113                 shutil.rmtree(path, ignore_errors=True)
114 
115         def healthcheck(self) -> Tuple[bool, str]:
116             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
117             if not self.cfg.weights_path:
118                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
119             weights = os.path.expanduser(self.cfg.weights_path)
120             if not os.path.exists(weights):
121                 return False, f"WASB weights not found at {weights}"
122             repo_src = self._repo_src()
123             if not os.path.isdir(repo_src):
124                 return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
125                                f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
126             code = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
127             try:
128                 res = self._run([self.cfg.python_bin, "-c", code])
129             except FileNotFoundError:
130                 return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
131             except subprocess.TimeoutExpired:
132                 return False, "native torch healthcheck timed out."
133             out = (res.stdout or "").strip()
134             if res.returncode != 0:
135                 return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
136             if self.cfg.device == "cuda" and "cuda True" not in out:
137                 return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
138             return True, out
139 
140         def run_predict(self, video_win_path: str, output_win_dir: str,
141                         video_id: Optional[str] = None) -> Optional[str]:
142             """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
143 
144             Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
145             runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
146             same downstream contract, so the parity comparison is apples-to-apples.
147             """
148             output_dir = os.path.abspath(output_win_dir)
149             os.makedirs(output_dir, exist_ok=True)
150             # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
151             norm = str(video_win_path).replace("\\", "/")
152             stem, _ext = os.path.splitext(os.path.basename(norm))
153             # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
154             key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
155             out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
156 
157             if not self._copy_infer_script():
158                 logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
159                 return None
```

```
160
161             weights = os.path.expanduser(self.cfg.weights_path)
162             argv = [self.cfg.python_bin, "wasb_infer.py",
163                     "--video", os.path.abspath(str(video_win_path))]
164             frames_dir: Optional[str] = None
165             if self.cfg.stream_video:
166                 # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
167                 argv.append("--stream-video")
168             else:
169                 scratch = self.cfg.scratch_dir or output_dir
170                 frames_dir = os.path.join(scratch, f"{key}_frames")
171                 argv += ["--frames_out_dir", frames_dir]
172             argv += ["--weights", weights,
173                      "--sport", self.cfg.sport,
174                      "--device", self.cfg.device,
175                      "--out", out_csv]
176
177             logger.info("Running native WASB inference (device=%s) on %s ...",
178                         self.cfg.device, os.path.basename(norm))
179             try:
180                 res = self._run(argv, cwd=self._repo_src())
181             except FileNotFoundError:
182                 logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
183                 return None
184             except subprocess.TimeoutExpired:
185                 logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
186                 return None
187             if res.returncode != 0:
188                 logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
189                 return None
190             if not os.path.exists(out_csv):
191                 logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
192                 return None
193             logger.info("WASB trajectory CSV: %s", out_csv)
194
195             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
196             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
197             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
198             if not self.cfg.keep_frames and frames_dir:
199                 logger.info("Cache hygiene: removing frame cache for %s "
200                             "(set indexing.wasb.keep_frames=true to retain).", key)
201                 self._rm_rf(frames_dir)
202             return out_csv
203
204         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
205         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
206         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
207
```

4. user (2026-06-20T08:57:15.577Z)

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
```

```
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
12      import os
13      import shlex
14      import logging
15      import subprocess
16      from dataclasses import dataclass
17      from typing import Dict, Any, Optional, Tuple
18
19      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20      # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21      from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24      logger = logging.getLogger(__name__)
25
26      # Windows path to the inference wrapper we stage into WSL.
27      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30      @dataclass
31      class WasbConfig:
32          """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/WASB-SBDT"
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "wasb"
37          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38          sport: str = "badminton"
39          wsl_stage_dir: str = "~/clips"
40          timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41          keep_frames: bool = False  # retain staged video + frame cache after success (debug
only)
42          min_free_gb: float = 0.0   # warn if WSL free space below this before a run (0 =
off)
43          # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44          # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45          # extraction that dominates a long clip. Output is bit-identical to the PNG path
46          # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47          stream_video: bool = False
48
49          @classmethod
50          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51              w = (idx_cfg or {}).get("wasb", {}) or {}
52              return cls(
53                  distro=w.get("wsl_distro", "Ubuntu"),
54                  repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55                  conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56                  conda_env=w.get("conda_env", "wasb"),
57                  weights_path=w.get("weights_path",
58                                     "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59                  sport=w.get("sport", "badminton"),
60                  wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61                  timeout_sec=int(w.get("timeout_sec", 1800)),
```

```
                keep_frames=bool(w.get("keep_frames", False)),
                min_free_gb=float(w.get("min_free_gb", 0.0)),
                stream_video=bool(w.get("stream_video", False)),
            )


    class WasbRunner(WslCommandMixin, DetectorRunner):
        def __init__(self, cfg: WasbConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
            cmd = (f"conda activate {self.cfg.conda_env} && "
                   f"python -c \"import torch; print('torch', torch.__version__, "
                   f"'cuda', torch.cuda.is_available())\"")
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? Windows + WSL2 required."
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
            return True, out

        def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        def _stage_infer_script(self) -> bool:
            """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
            src_mnt = to_wsl_mnt_path(_INFER_WIN)
            cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
            res = self._wsl_bash(cmd)
            return res.returncode == 0 and "OK" in (res.stdout or "")

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.

            All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
            two videos that share a filename (e.g. two ``match.mp4`` from different folders)
            can never reuse each other's staged frames, resumable cache, or CSV.
            """
            # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
            # translation: to_wsl_mnt_path passes relative paths through unchanged, so
            # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
```

```
            # the CSV landed inside WSL while Windows polled the repo-relative path.
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize for cross-platform basename (tests use Windows paths on Linux).
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)
            # Unique, content-derived key: same filename + different bytes -> different key.
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            wsl_out_dir = to_wsl_mnt_path(output_win_dir)
            expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")

            if not self._stage_infer_script():
                logger.error("Could not stage wasb_infer.py into WSL.")
                return None
            staged_video = self._stage_video(video_win_path, f"{key}{ext}")
            if staged_video is None:
                return None
            frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"

            # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
            # WSL stage filesystem is already low so the user can clean up before it fills.
            # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
            if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning(
                        "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
                        "may fill the disk ? consider running tools/cleanup_caches.py.",
                        free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            # Fast path: stream-decode the video in-process (no PNG extraction). Output is
            # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
            if self.cfg.stream_video:
                frame_args = "--stream-video "
            else:
                frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
            cmd = (
                f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
                f"python wasb_infer.py "
                f"--video {wsl_tilde_quote(staged_video)} "
                f"{frame_args}"
                f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--sport {shlex.quote(self.cfg.sport)} "
                f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
            )
            logger.info("Running WASB inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
                return None
            if res.returncode != 0:
                logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None
            if not os.path.exists(expected_csv_win):
                logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
                return None
            logger.info("WASB trajectory CSV: %s", expected_csv_win)
```

```
184             # Success ? the trajectory CSV is the durable output; the staged video copy and
185             # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186             # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187             # so the frame cache + detector cache remain for a resumable re-run.
188             # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189             if not self.cfg.keep_frames:
190                 stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191                 logger.info("Cache hygiene: removing staged video%s for %s "
192                             "(set indexing.wasb.keep_frames=true to retain).",
193                             "" if self.cfg.stream_video else " + frame cache", key)
194                 self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200
```

5. assistant (2026-06-20T08:57:19.939Z)

Let me read the rest of the changed files in the working tree.

6. user (2026-06-20T08:57:20.905Z)

```
1       """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3       This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4       of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5       the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7       The ABC lets us:
8       - Keep the current WSL adapters working unchanged for Windows users.
9       - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10        the hybrid segmenters or the model-agnostic windowing brain.
11      - Provide a single place to evolve the healthcheck / error contract and thread
12        detector readiness into per-video reports.
13
14      Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15      - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16        Run detector, return the Windows-side path to the produced trajectory CSV
17        (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18        The caller (hybrid) is responsible for output dir; impls may stage the video
19        into WSL fs for perf.
20
21      - healthcheck() -> tuple[bool, str]
22        Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23        Message is logged (and can be surfaced to reports or UX in future).
24        Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26      Existing behavior is 100% preserved. The only additions are the inheritance and
27      the explicit common type for future implementers.
28
29      How to add a new runner (for docs / future contributors):
30      1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31      2. Implement run_predict and healthcheck (return (bool, str) tuple).
32      3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33         TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34         deliberately static and model-agnostic). If the new detector needs different
35         windowing, it can implement its own or we generalize later.
36      4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37         registry never pulls torch/tf on a machine that doesn't have them).
```

```
    5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
       DetectorRunner instances (they already do the right thing via duck-typing + the
       type hints we add here).
    6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
       returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
    7. No change to eval/calibration code paths (they consume the CSV + the static
       windowing helpers directly).

    Optional Linux-native path (the main de-risk goal):
    A LinuxNativeRunner (or ONNXRunner) would:
    - Skip all wsl / _stage / conda activate.
    - Load weights locally (torch or onnx) or call a native binary.
    - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
    - healthcheck can just check "weights file exists and torch importable + CUDA?".
    This removes the WSL requirement for Linux users / future SaaS workers while
    keeping the exact same candidate quality characteristics (or better, once native
    weights are available).

    Error surfacing into reports:
    The healthcheck message is already logged by the hybrids on failure. The reporting
    harvest sees the outcome as "0 candidates" + detector name. When we wire richer
    segmenter Results (or pass detector_health into emit_indexer_report context), the
    (ok, msg) can be recorded explicitly under segmentation.details or a new detector
    stage. The ABC makes that future change local to one place.

    See also:
    - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
    - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
    - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
    """

    import logging
    import shlex
    import subprocess
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional, Tuple

    logger = logging.getLogger(__name__)


    def wsl_tilde_quote(path: str) -> str:
        """shlex.quote that still lets bash expand a leading ``~/``.

        ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
        treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
        directory" ? this broke live WSL staging for the default ``~/clips``
        stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
        "shlex-quote breaks ~ expansion" item). Quoting only the part after
        ``~/`` keeps expansion AND protects spaces/specials in the remainder.
        """
        if path == "~":
            return path
        if path.startswith("~/"):
            return "~/" + shlex.quote(path[2:])
        return shlex.quote(path)


    class WslCommandMixin:
        """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.

        Lives beside (not on) DetectorRunner: a future Linux-native runner must not
        inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
        and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
        ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
        """
```

```
103
104         cfg: Any
105
106         def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107             full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108             argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109             logger.debug("WSL exec: %s", full)
110             return subprocess.run(argv, capture_output=True, text=True,
111                                   timeout=(self.cfg.timeout_sec or None))
112
113         # ---- cache hygiene helpers (shared by both WSL runners) ------------------
114         def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115             """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.
116
117             Returns None if it can't be determined (never raises). Used as a pre-run
118             disk guard so a long extraction doesn't fill the disk mid-flight.
119             """
120             try:
121                 res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
'NR==2{{print $4}}'")
122             except (subprocess.TimeoutExpired, OSError):
123                 return None
124             try:
125                 return int((res.stdout or "").strip()) / (1024 ** 3)
126             except (ValueError, AttributeError):
127                 return None
128
129         def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
130             """Best-effort recursive delete of WSL paths (post-success cleanup; never
raises)."""
131             paths = [p for p in wsl_paths if p]
132             if not paths:
133                 return
134             quoted = " ".join(wsl_tilde_quote(p) for p in paths)
135             try:
136                 self._wsl_bash(f"rm -rf {quoted}")
137             except (subprocess.TimeoutExpired, OSError) as e:
138                 logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
139
140         def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
141             """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
142
143             Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
144             filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
145             fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
146             trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
147             A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
any
148             frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
149             ``cp`` still gets its chance (we never block a genuinely-readable source).
150             """
151             try:
152                 res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
153             except (subprocess.TimeoutExpired, OSError):
154                 return None
155             if "READABLE" in (res.stdout or ""):
156                 return None
157             return (
158                 f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
159                 f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
160                 f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
161             )
162
163
164     class DetectorRunner(ABC):
```

```
165         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
166
167         @abstractmethod
168         def run_predict(self, video_win_path: str, output_win_dir: str,
169                         video_id: Optional[str] = None) -> Optional[str]:
170             """Run inference and return path to trajectory CSV (or None on failure).
171
172             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
173             videos that happen to share a filename cannot reuse each other's frames or CSV.
174             It is optional for back-compat; callers in the live pipeline always pass it.
175             """
176             pass
177
178         @abstractmethod
179         def healthcheck(self) -> Tuple[bool, str]:
180             """Quick check if the runner (env, weights, etc.) is usable.
181
182             Returns (ok, message). Never raises. Cheap to call before a long run.
183             """
184             pass
185
186         # De facto trajectory contract: the trajectory hybrids call these
187         # unconditionally after run_predict. Concrete runners reuse/re-export the
188         # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189         # a runner with custom windowing supplies its own with the same shapes.
190         parse_trajectory_csv: Callable[..., Any]
191         trajectory_to_action_windows: Callable[..., Any]
192
```

7. user (2026-06-20T08:57:21.976Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
```

```
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False                       # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)
44
45          @classmethod
46          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47              tn = (idx_cfg or {}).get("tracknet", {}) or {}
48              return cls(
49                  distro=tn.get("wsl_distro", "Ubuntu"),
50                  repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51                  conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52                  conda_env=tn.get("conda_env", "TrackNetV4"),
53                  weights_path=tn.get("weights_path", ""),
54                  queue_length=int(tn.get("queue_length", 5)),
55                  stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56                  wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57                  timeout_sec=int(tn.get("timeout_sec", 1800)),
58                  keep_frames=bool(tn.get("keep_frames", False)),
59                  min_free_gb=float(tn.get("min_free_gb", 0.0)),
60              )
61
62
63      def to_wsl_mnt_path(win_path: str) -> str:
64          """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66          Already-POSIX paths (starting with / or ~) are returned unchanged so the
67          same helper is safe to call on either kind of path.
68          """
69          p = str(win_path)
70          if p.startswith("/") or p.startswith("~"):
71              return p
72          p = p.replace("\\", "/")
73          if len(p) >= 2 and p[1] == ":":
74              drive = p[0].lower()
75              return f"/mnt/{drive}{p[2:]}"
76          return p
77
78
79      @dataclass
80      class TrajectoryPoint:
81          frame: int
82          visible: bool
83          x: float
84          y: float
85
86
87      class TrackNetRunner(WslCommandMixin, DetectorRunner):
88          """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90          Implements the common DetectorRunner contract so a future Linux-native or
91          alternative trajectory runner can be substituted without touching the hybrids.
```

```
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
            therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
            a filename cannot collide on the output CSV.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
            # relative paths through unchanged, so WSL would resolve them against the
            # predict.py cwd instead of the repo (same bug class as wasb_runner).
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
```

```
            # Namespace by video_id so two same-filename videos don't collide on the CSV.
            # predict.py derives its output name from the (staged) video stem, so staging
            # under the namespaced name propagates into "<key>_predict.csv".
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, f"{key}{ext}")
                if staged is None:
                    return None
                wsl_video = staged
            else:
                # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
                # We cannot rename its output, so fall back to the un-namespaced name.
                wsl_video = to_wsl_mnt_path(video_win_path)
                expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            if self.cfg.min_free_gb > 0:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
                                   "consider running tools/cleanup_caches.py.",
                                   free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"cd {self.cfg.repo_dir}/src && "
                f"python predict.py "
                f"--video_path {wsl_tilde_quote(wsl_video)} "
                f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--output_dir {wsl_tilde_quote(wsl_out)} "
                f"--queue_length {self.cfg.queue_length}"
            )
            logger.info("Running TrackNet inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
                return None

            if res.returncode != 0:
                logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None

            if not os.path.exists(expected_csv_win):
                logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
                return None
            logger.info("TrackNet trajectory CSV: %s", expected_csv_win)

            # Success ? the CSV is the durable output; drop the staged video copy (a pure,
            # regenerable intermediate). On earlier failure we return above without
cleaning.
            if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
                logger.info("Cache hygiene: removing staged video for %s "
                            "(set indexing.tracknet.keep_frames=true to retain).", key)
                self._wsl_rm_rf([wsl_video])
            return expected_csv_win

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
```

```
214
215             Returns the WSL path to the staged copy, or None on failure.
216             """
217             src_mnt = to_wsl_mnt_path(video_win_path)
218             reason = self.wsl_source_unreadable_reason(src_mnt)
219             if reason:
220                 logger.error("Cannot stage video into WSL: %s", reason)
221                 return None
222             dest = f"{self.cfg.wsl_stage_dir}/{base}"
223             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224             try:
225                 res = self._wsl_bash(cmd)
226             except subprocess.TimeoutExpired:
227                 logger.error("Staging video into WSL timed out.")
228                 return None
229             if res.returncode != 0 or "OK" not in (res.stdout or ""):
230                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231                 return None
232             return dest
233
234         # ------------------------------------------------------------ CSV -> windows
235
236         @staticmethod
237         def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238             """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239             points: List[TrajectoryPoint] = []
240             with open(csv_win_path, newline="") as f:
241                 reader = csv.DictReader(f)
242                 for row in reader:
243                     try:
244                         points.append(TrajectoryPoint(
245                             frame=int(row["Frame"]),
246                             visible=int(row["Visibility"]) == 1,
247                             x=float(row["X"]),
248                             y=float(row["Y"]),
249                         ))
250                     except (KeyError, ValueError):
251                         continue
252             points.sort(key=lambda p: p.frame)
253             return points
254
255         @staticmethod
256         def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
257                            velocity_thresh: float) -> List[Tuple[int, float]]:
258             """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
259
260             Computes each visible point's normalised velocity from the last *visible* point;
261             an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262             ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
263             the caller so identical floats feed the velocity math."""
264             active = []  # (frame_idx, velocity)
265             prev: Optional[TrajectoryPoint] = None
266             for p in points:
267                 if not p.visible:
268                     prev = None  # break the trajectory; absent shuttle = not in flight
269                     continue
270                 if prev is not None:
271                     dframes = p.frame - prev.frame
272                     if dframes > 0:
273                         dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
```

```
                        velocity = (dist / frame_width) * (fps / dframes)
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
                prev = p
            return active

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
            max_window_duration: float = 0.0,
            min_split_gap: float = 0.5,
            window_hard_cap: bool = False,
            window_inactivity_end: float = 0.0,
            inactivity_min_density: float = 0.34,
            inactivity_rally_thresh: float = 0.08,
            window_blob_fallback: bool = False,
            window_blob_factor: float = 4.0,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.

            ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
            the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
            and a single window swallows several rallies + the dead time between them. When
set, any
            window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
            longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
            that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
            ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
            drop, and it is config-gated default-OFF until measured on the golden set.

            ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
            over-long window has NO internal inactivity gap to split on (a continuously-
active
            trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
            blob where the gap-splitter alone left a single 174 s window), it is recursively
            hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts

            (never merges), so recall cannot drop; gated default-OFF until measured.

            ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
            ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
```

```
326             over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
327             force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` ×
328             ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329             their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
330             alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331             each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332             rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333             can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334             """
335             if fps <= 0:
336                 fps = 30.0
337             if frame_width <= 0:
338                 frame_width = 1280.0
339
340             active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342             if not active:
343                 return []
344
345             max_gap_frames = int(merge_gap * fps)
346
347             def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348                 vels = [v for _, v in group]
349                 return {
350                     "start": group[0][0] / fps + chunk_start,
351                     "end": group[-1][0] / fps + chunk_start,
352                     "active_frame_count": len(group),
353                     "peak_velocity": max(vels),
354                     "mean_velocity": sum(vels) / len(vels),
355                     "track_id_count": 1,  # single shuttle; kept for window-shape parity
356                     "chunk_index": chunk_index,
357                 }
358
359             groups: List[List[Tuple[int, float]]] = []
360             group = [active[0]]
361             for af in active[1:]:
362                 if af[0] - group[-1][0] <= max_gap_frames:
363                     group.append(af)
364                 else:
365                     groups.append(group)
366                     group = [af]
367             groups.append(group)
368
369             if max_window_duration > 0:
370                 groups = TrackNetRunner._split_overlong_groups(
371                     groups, int(max_window_duration * fps), int(min_split_gap * fps),
372                     hard_cap=window_hard_cap)
373
374             # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
375             # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
376             # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
377             # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
```

```
            # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
            if window_inactivity_end > 0:
                tw = int(window_inactivity_end * fps)
                groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
                                                             inactivity_rally_thresh) for g
in groups]

            # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
            # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
            # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
            # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
            forced_flags = [False] * len(groups)
            if window_blob_fallback and max_window_duration > 0:
                groups, forced_flags = TrackNetRunner._blob_fallback_chop(
                    groups, int(max_window_duration * fps), window_blob_factor)

            windows = []
            for g, forced in zip(groups, forced_flags):
                w = _finalize(g)
                if forced:
                    w["forced_cut"] = True
                windows.append(w)
            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

        @staticmethod
        def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
                                min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
            """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
            (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
            over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
            (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
            ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
            LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
            can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
            exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
            is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
            if trail_frames <= 0 or len(group) < 2:
                return group
            frames = [f for f, _ in group]
            fast = [1 if v > rally_thresh else 0 for _, v in group]
            lo = 0
            fast_sum = 0
            last_dense = -1
            for i in range(len(frames)):
                fast_sum += fast[i]
                while frames[lo] <= frames[i] - trail_frames:
                    fast_sum -= fast[lo]
                    lo += 1
                win_count = i - lo + 1
                if win_count > 0 and fast_sum / win_count >= min_density:
                    last_dense = i
```

```
            return group[: last_dense + 1] if last_dense >= 0 else group

        @staticmethod
        def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
            """Index of the active frame nearest the group's temporal midpoint (used to chop
a
            gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
            mid = (group[0][0] + group[-1][0]) / 2.0
            split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
            return split_i

        @staticmethod
        def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
                                   min_split_frames: int,
                                   hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
            """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
            internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
            boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
            invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
            rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
            temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
            bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
            if max_win_frames <= 0:
                return groups
            out: List[List[Tuple[int, float]]] = []
            work = list(groups)
            while work:
                g = work.pop()
                if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
                    out.append(g)
                    continue
                best_i, best_gap = -1, -1
                for i in range(1, len(g)):
                    gap = g[i][0] - g[i - 1][0]
                    if gap > best_gap:
                        best_gap, best_i = gap, i
                if best_i < 0 or best_gap < min_split_frames:
                    # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
                    # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
                    if hard_cap:
                        split_i = TrackNetRunner._midpoint_split_index(g)
                        work.append(g[:split_i])
                        work.append(g[split_i:])
                    else:
                        out.append(g)
                    continue
                work.append(g[:best_i])
                work.append(g[best_i:])
            out.sort(key=lambda grp: grp[0][0])
            return out

        @staticmethod
        def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
                                blob_factor: float = 4.0
```

```
481                                ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482             """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483             inactivity trim have each had their chance, a group still longer than
``blob_factor`` ×
484             ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
485             signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
486             11× a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2× the cap)
487             is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
488             midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
489             degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
490             rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
491             can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
492             that piece came from a forced chop. Iterative worklist to bound stack depth."""
493             if max_win_frames <= 0:
494                 return groups, [False] * len(groups)
495             blob_frames = max_win_frames * max(1.0, blob_factor)
496             out: List[List[Tuple[int, float]]] = []
497             flags: List[bool] = []
498             forced = 0
499             for g in groups:
500                 if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
501                     out.append(g)
502                     flags.append(False)
503                     continue
504                 pieces: List[List[Tuple[int, float]]] = []
505                 work = [g]
506                 while work:
507                     h = work.pop()
508                     if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
509                         pieces.append(h)
510                         continue
511                     split_i = TrackNetRunner._midpoint_split_index(h)
512                     work.append(h[:split_i])
513                     work.append(h[split_i:])
514                 pieces.sort(key=lambda grp: grp[0][0])
515                 out.extend(pieces)
516                 flags.extend([True] * len(pieces))
517                 forced += len(pieces) - 1
518             if forced:
519                 logger.warning(
520                     "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
521                     "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
522                     "motion), not a bigger cap", forced)
523             return out, flags
524
```

8. assistant (2026-06-20T08:57:24.114Z)

Now let me read the segmenter files and wasb_infer.py.

9. user (2026-06-20T08:57:25.077Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
```

```
    standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
    now the single home for the whole pipeline shape:

        healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
        -> P3 AI-provider handoff over padded clips -> P4 DB population/linking

    A concrete subclass supplies only its identity and runner wiring:
    - ``family``        ? config section under ``indexing.*``, output subdir, and
                          the ``<family>-hybrid-<model>`` detector tag in metadata.
    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
    - ``name`` / ``description`` properties (the registry contract).
    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
                          detection_params extras such as weights_path).

    This base is deliberately NOT registered; only concrete subclasses register.
    """
    import os
    import time
    import logging
    import concurrent.futures
    from typing import Any, Dict, List, Optional, Tuple

    import cv2

    from backend.pipeline.segmenters.base import BasePlaySegmenter
    from backend.utils.video import get_video_duration, extract_video_chunk
    from backend.sports import sport_registry
    from backend.providers import provider_registry
    from backend.pipeline.detectors.base import DetectorRunner

    logger = logging.getLogger(__name__)


    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class TrajectoryHybridSegmenter(BasePlaySegmenter):
        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""

        #: config section under `indexing` (velocity_threshold lives there), the
        #: output subdir for trajectories, and the metadata detector-tag prefix.
        family: str = ""
        #: human-readable label used in log lines.
        display_label: str = ""

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
            raise NotImplementedError

        def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
            """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
            native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
            raise NotImplementedError(
                f"detector_impl='native' is not supported for the "
                f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
```

```
has a "
63                  f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
64
65          def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
66              """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
67
68              ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
69              ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70              is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71              in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72              ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73              (the WSL runner) so the default production path is unchanged
74              (``docs/DECOUPLED_COMPUTE/``).
75              """
76              impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77              if impl == "stub":
78                  from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79                  logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                              self.display_label or "trajectory")
81                  return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82              if impl in ("native", "native_wasb", "wasb_native"):
83                  logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                              self.display_label or "trajectory")
85                  return self._build_native_runner(idx_cfg)
86              return self._build_runner(idx_cfg)
87
88          @staticmethod
89          def _probe_fps_width(video_path: str) -> tuple:
90              """Return (fps, frame_width) for a video, with sensible fallbacks."""
91              cap = cv2.VideoCapture(video_path)
92              fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93              width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94              cap.release()
95              return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97          @staticmethod
98          def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99              """Provider-reported exchange count, else a duration-based estimate.
100
101             One canonical implementation ? the twins had drifted (wasb relied on
102             ``int(None)`` raising into the fallback; same result, by accident).
103             """
104             shot_count = segment.get("shuttle_exchange_count")
105             if shot_count is None:
106                 return max(3, int(duration / 0.8))
107             try:
108                 return int(shot_count)
109             except (ValueError, TypeError):
110                 return max(3, int(duration / 0.8))
111
112         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
113         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
114                                     frame_width: float, video_path: str,
115                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
116             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
```

```
BASE
117             returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
118             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
119             features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
120             return {
121                 "peak_velocity": cw["peak_velocity"],
122                 "mean_velocity": cw["mean_velocity"],
123                 "active_frame_count": cw["active_frame_count"],
124                 "track_id_count": cw["track_id_count"],
125             }
126
127         def _select_for_handoff(self, windows: List[Dict[str, Any]],
128                                 window_stats: List[Dict[str, Any]],
129                                 idx_cfg: Dict[str, Any]) -> List[int]:
130             """Indices of the candidate windows that proceed to the AI handoff (and thus may
131             become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
132             ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
133             Persisted candidates are NOT affected ? they remain the full superset for
offline
134             re-scoring."""
135             return list(range(len(windows)))
136
137         def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
138                           player_pool: Optional[List[str]] = None,
139                           max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
140             # override-ignore: the ABC declares the Result contract (Success|Failure)
141             # but every live segmenter still returns a bare list ? the documented
142             # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
143             label = self.display_label
144             # --- Sport profile + AI provider wiring (identical across segmenters) ---
145             sport_name = self.config.get("sport", "badminton")
146             try:
147                 sport_profile = sport_registry.get(sport_name)
148             except KeyError as e:
149                 logger.error(str(e))
150                 return []
151
152             idx_cfg = self.config.get("indexing", {})
153             # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
154             # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
155             # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
156             # measure it against the Gemini path, or to avoid the cloud entirely). With
157             # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
158             skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
159             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
160             if skip_ai:
161                 model_name = "local-noai"
162                 logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
163                             "be emitted as rallies; no %s handoff, no API key needed.",
164                             label, provider_name)
165             else:
166                 model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
167                 api_key_env_var = f"{provider_name.upper()}_API_KEY"
168                 api_key = os.environ.get(api_key_env_var)
169                 if not api_key:
170                     from backend.pipeline.segmenters._keyhint import api_key_hint
```

```
                    logger.error(
                        f"{api_key_env_var} environment variable is not set! "
                        f"To run the {provider_name} handoff, set your API key. "
                        + api_key_hint(api_key_env_var)
                    )
                    return []
                try:
                    provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
                except KeyError as e:
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
            # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
            # it delegates to the subclass _build_runner (the real WSL/native runner).
            runner, runner_params = self._resolve_runner(idx_cfg)

            velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
            max_window_duration = idx_cfg.get("max_window_duration", 0.0)
            window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
            window_hard_cap = idx_cfg.get("window_hard_cap", False)
            window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
            inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
            inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
            window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
            window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s detector environment not ready: %s", label, msg)
                return []
            logger.info("%s detector env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
```

```
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
                max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
                window_hard_cap=window_hard_cap,
                window_inactivity_end=window_inactivity_end,
                inactivity_rally_thresh=inactivity_rally_thresh,
                inactivity_min_density=inactivity_min_density,
                window_blob_fallback=window_blob_fallback,
                window_blob_factor=window_blob_factor,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)

```

```
            ai_segments: List[dict] = []
            if skip_ai:
                # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
                # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
                # falls back to the duration-based estimate (no AI exchange count).
                ai_segments = [
                    {
                        "start_time": all_candidate_windows[wi]["start"],
                        "end_time": all_candidate_windows[wi]["end"],
                        "shuttle_exchange_count": None,
                        "shot_count_confidence": "LocalNoAI",
                        "_candidate_id": candidate_ids[wi],
                    }
                    for wi in handoff_indices
                ]
                logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
                            "(no AI handoff).", len(ai_segments))
            else:
                handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
                os.makedirs(handoff_dir, exist_ok=True)
                logger.info("Phase 3: %s validation on %d candidate windows...",
                            provider_name, len(handoff_indices))

                def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                    w_start = max(0, window["start"] - handoff_padding)
                    w_end = min(video_duration, window["end"] + handoff_padding)
                    w_dur = w_end - w_start
                    w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                    if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                        return []
                    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                    segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
                                                         label=f"Handoff {wi+1}")
                    valid = []
                    for seg in segments:
                        try:
                            seg["start_time"] = float(seg["start_time"]) + w_start
                            seg["end_time"] = float(seg["end_time"]) + w_start
                            seg["_candidate_id"] = candidate_ids[wi]
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
                            logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid

                with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                    futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
                               for wi in handoff_indices]
                    for future in futures:
                        ai_segments.extend(future.result())
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
```

```
346                 logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
347
348             # --- Phase 4: DB population + candidate linking ---
349             segments_data = []
350             for segment in ai_segments:
351                 try:
352                     start = float(segment["start_time"])
353                     end = float(segment["end_time"])
354                     dur = end - start
355                     metadata = {
356                         "shot_count": self._shot_count_from(segment, dur),
357                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
358                         "detector": f"{self.family}-hybrid-{model_name}",
359                     }
360                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
361                     if segment_id:
362                         candidate_id = segment.get("_candidate_id")
363                         if candidate_id is not None:
364                             self.db.link_candidate_to_segment(candidate_id, segment_id)
365                         segments_data.append({
366                             "id": segment_id, "start_time": start, "end_time": end,
367                             "duration": dur, "metadata": metadata,
368                         })
369                 except Exception as e:
370                     logger.warning("Skipping invalid segment struct: %s", e)
371
372             return segments_data
373
```

10. user (2026-06-20T08:57:26.082Z)

```
1       """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3       WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4       own building blocks (ImageDataset transform + detector + online tracker) to run on
5       any folder of extracted frames and emit a trajectory CSV in the exact shape the
6       indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8           Frame,Visibility,X,Y
9
10      It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11      imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12      (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14      Resumable across reboots
15      ------------------------
16      The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17      the slow, expensive part; the CPU tracker pass that turns detections into a
18      trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19      resume mid-stream). We exploit that split:
20
21        * The detector pass writes its per-window raw outputs incrementally to
22          ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23          records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24          most ``--flush-every`` windows of GPU work instead of the whole pass.
25        * On restart we replay the cached windows, skip the DataLoader ahead to the first
26          un-cached window, and continue. Once every window is cached, the detector pass
27          is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28          the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29          not the GPU hour.
30
31      The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
```

```
    to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
    weights, sport, window size, or frame count change.

    Usage (in WSL, from ~/models/WASB-SBDT/src):
        python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
            --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
    """
    import os
    import os.path as osp
    import csv
    import glob
    import json
    import argparse
    import logging
    from collections import defaultdict
    from contextlib import contextmanager

    logger = logging.getLogger(__name__)

    CACHE_VERSION = 1
    _DET_FILE = "det_raw.jsonl"
    _MANIFEST_FILE = "manifest.json"


    def _build_cfg(weights: str, sport: str, device: str = "cuda"):
        """Compose the WASB eval config via Hydra, overriding model/weights/device.

        ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
preserved),
        ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
``runner.gpus=[0]``;
        cpu/mps run with no GPU list so the detector places tensors on ``device``.
        """
        from hydra import compose, initialize
        gpus = "[0]" if device == "cuda" else "[]"   # single GPU on cuda; none for cpu/mps
        with initialize(config_path="configs", version_base=None):
            cfg = compose(config_name="eval", overrides=[
                f"dataset={sport}",
                "model=wasb",
                f"detector.model_path={weights}",
                f"runner.device={device}",
                f"runner.gpus={gpus}",  # default config assumes multiple GPUs; pin to this
box
            ])
        return cfg


    def _frame_id(path: str) -> int:
        """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
        stem = osp.splitext(osp.basename(path))[0]
        digits = "".join(ch for ch in stem if ch.isdigit())
        return int(digits) if digits else -1


    # ---------------------------------------------------------------------- #
    # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
    # ---------------------------------------------------------------------- #
    def _cache_paths(cache_dir: str):
        return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)


    def default_cache_dir(out_csv: str) -> str:
```

```
91          """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
92          d = osp.dirname(osp.abspath(out_csv))
93          stem = osp.splitext(osp.basename(out_csv))[0]
94          return osp.join(d, stem + "_wasbcache")
95
96
97      def _atomic_write_text(path: str, text: str) -> None:
98          """Write then rename so a crash mid-write can't leave a half-written file."""
99          tmp = path + ".tmp"
100         with open(tmp, "w") as f:
101             f.write(text)
102             f.flush()
103             os.fsync(f.fileno())
104         os.replace(tmp, path)
105
106
107     def _det_plain(det) -> list:
108         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
109         xy = det["xy"]
110         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
111
112
113     def _dets_to_tracker(plain_list):
114         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
115         the tracker does vector math / np.linalg.norm on it)."""
116         import numpy as np
117         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
118                 for p in plain_list]
119
120
121     def write_manifest(cache_dir: str, manifest: dict) -> None:
122         _, mpath = _cache_paths(cache_dir)
123         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
124
125
126     def read_manifest(cache_dir: str):
127         _, mpath = _cache_paths(cache_dir)
128         if not osp.exists(mpath):
129             return None
130         try:
131             with open(mpath) as f:
132                 return json.load(f)
133         except (json.JSONDecodeError, OSError):
134             return None
135
136
137     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
138                             frames_in: int, frames_total: int, device: str = "cuda") ->
bool:
139         """A cache is reusable only if the run that produced it matches this run's inputs.
140
141         ``device`` defaults to ``cuda`` so caches written before device-keying (no
``device``
142         field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
cache
143         (detections can differ numerically across devices), and vice-versa.
144         """
145         if not manifest or manifest.get("version") != CACHE_VERSION:
146             return False
147         return (
148             manifest.get("frames_dir") == osp.abspath(frames_dir)
149             and manifest.get("weights") == weights
150             and manifest.get("sport") == sport
```

```
            and manifest.get("frames_in") == frames_in
            and manifest.get("frames_total") == frames_total
            and manifest.get("device", "cuda") == device
        )


    def reset_cache(cache_dir: str) -> None:
        """Drop any existing cache so the next run starts clean."""
        det_jsonl, mpath = _cache_paths(cache_dir)
        for p in (det_jsonl, mpath):
            if osp.exists(p):
                os.remove(p)


    def load_and_clean_cache(cache_dir: str):
        """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
        rebuild the per-frame detection accumulation, and rewrite the file to exactly that
        prefix so a trailing half-written line from a crash can't corrupt future appends.

        Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
        [x, y, score, scale] candidates (accumulated across every window the frame is in,
        in window order ? identical to a non-resumed run).
        """
        det_jsonl, _ = _cache_paths(cache_dir)
        det_by_fid = defaultdict(list)
        valid: list = []
        if osp.exists(det_jsonl):
            with open(det_jsonl, "r") as f:
                for line in f:
                    s = line.strip()
                    if not s:
                        continue
                    try:
                        obj = json.loads(s)
                    except json.JSONDecodeError:
                        break                       # truncated trailing line (crash mid-
flush)
                    if obj.get("w") != len(valid):  # non-contiguous -> stop at the gap
                        break
                    valid.append(s)
                    for fid, plain in obj["f"]:
                        det_by_fid[int(fid)].extend([list(p) for p in plain])
            # Guarantee a clean append boundary by rewriting only the good prefix.
            _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
        return len(valid), det_by_fid


    def _append_windows(fh, objs) -> None:
        """Append window records as JSONL and durably flush (bounded loss on crash)."""
        for o in objs:
            fh.write(json.dumps(o, separators=(",", ":")) + "\n")
        fh.flush()
        os.fsync(fh.fileno())


    def _atomic_write_trajectory(out_csv: str, rows) -> None:
        os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
        tmp = out_csv + ".tmp"
        with open(tmp, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["Frame", "Visibility", "X", "Y"])
            w.writerows(rows)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, out_csv)
```

```
215
216
217     # ---------------------------------------------------------------------------- #
218     # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
219     # ---------------------------------------------------------------------------- #
220     _STREAM_FRAME_FMT = "{:08d}.png"
221
222
223     def synthetic_frame_path(index: int) -> str:
224         """Stable, index-encoding frame name used by the streaming path.
225
226         It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
227         downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
228         frames came from disk or from the in-memory stream. The streaming image loader
229         inverts this name back to an index to fetch the decoded frame.
230         """
231         return _STREAM_FRAME_FMT.format(int(index))
232
233
234     def build_windows(frames: list, frames_in: int):
235         """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
236         + checkpoint). Pure list math, identical for the disk and streaming paths.
237
238         Returns a list of windows, each a list of ``frames_in`` consecutive paths
239         (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
240         caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
241         ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
242         """
243         return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
244
245
246     class SequentialFrameStore:
247         """In-memory sliding buffer over a forward-only frame reader, sized for the
248         detector's sliding-window access pattern (peak O(window) frames, not O(video)).
249
250         The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
251         ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
252         in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
253         start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
254         boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
255         far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
256         earliest index any not-yet-finished window can still ask for) and decode each source
257         frame exactly once via the forward-only ``reader(index)``.
258         """
259
260         def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
261             self._reader = reader
262             self._total = int(total)
263             self._window = max(1, int(window))
264             self._buf: dict = {}
265             # On a resumed run the detector's first needed frame is `start_index`; begin
266             # pulling there so the reader can skip (seek past) the already-cached prefix
267             # instead of decoding 0..start_index-1 just to throw them away.
268             self._next = int(start_index)       # next index not yet pulled from the reader
269             self._max_seen = int(start_index) - 1
270             self._floor = int(start_index)      # lowest index we still keep (never evict
below)
271
272         def get(self, index: int):
273             index = int(index)
274             if index < self._floor:
275                 raise KeyError(
276                     f"frame {index} already evicted (below floor {self._floor}); the access
"
277                     f"pattern must stay within a window of the highest frame seen")
```

```
278             if index >= self._total:
279                 raise KeyError(f"frame {index} out of range (total {self._total})")
280             # Pull forward from the reader until the requested index is buffered.
281             while self._next <= index:
282                 self._buf[self._next] = self._reader(self._next)
283                 self._next += 1
284             if index > self._max_seen:
285                 self._max_seen = index
286             # Evict frames older than the earliest a still-running window could re-request:
287             # once we've seen index `m`, no future window starts before `m - (window-1)`.
288             new_floor = max(self._floor, self._max_seen - (self._window - 1))
289             for k in range(self._floor, new_floor):
290                 self._buf.pop(k, None)
291             self._floor = new_floor
292             return self._buf[index]
293
294
295     def _index_from_synthetic_path(path: str) -> int:
296         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
297
298         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
299         in lockstep with the on-disk naming scheme.
300         """
301         return _frame_id(path)
302
303
304     def _bgr_to_pil_rgb(frame_bgr):
305         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
306         PIL RGB image WASB's disk path produces.
307
308         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
309         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
310         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
311         frame indistinguishable from the on-disk one to everything downstream.
312         """
313         import cv2
314         from PIL import Image
315         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
316
317
318     def _make_streamed_read_image(store, real_read_image):
319         """Build the ``read_image`` replacement used during a streaming detector pass.
320
321         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
322         matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
323         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
324         without the WSL-only ``dataloaders`` package.
325         """
326         def _read_image(path, *args, **kwargs):
327             idx = _index_from_synthetic_path(path)
328             if idx < 0:
329                 return real_read_image(path, *args, **kwargs)
330             return _bgr_to_pil_rgb(store.get(idx))
331         return _read_image
332
333
334     @contextmanager
335     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
```

```
int = 0):
336         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
337         frames during the detector pass, byte-for-byte as it would over real PNGs.
338
339         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
340         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
341         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
342         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
343         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
344         ``osp.exists`` guard is sidestepped for synthetic stream paths.
345
346         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
347         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
348         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
349         reference).
350
351         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
352         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
353         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
354         original reader is restored on exit.
355         """
356         from dataloaders import dataset_loader as _dl
357         store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
358         real_read_image = _dl.read_image
359         # Rebind read_image in the consuming module for the duration of the detector pass;
360         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
361         _dl.read_image = _make_streamed_read_image(store, real_read_image)  # type:
ignore[assignment]
362         try:
363             yield store
364         finally:
365             _dl.read_image = real_read_image  # type: ignore[assignment]
366
367
368     # ------------------------------------------------------------------------ #
369     # Inference
370     # ------------------------------------------------------------------------ #
371     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
372             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
373             log_every_batches: int = 50, cache_dir: "str | None" = None,
374             flush_every: int = 1000, fresh: bool = False,
375             frame_loader=None, source_key: "str | None" = None,
376             device: str = "cuda") -> str:
377         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
378
379         Two equivalent ways to supply pixels (output is bit-identical between them):
380
381         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
382           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
383         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
384           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
385           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
```

```
386           reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
387           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
388           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
389           ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
``num_workers=0``
390           (the in-process shim + buffer can't cross worker processes).
391
392         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
393         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
394         after reboot still matches.
395         """
396         import time
397         import torch
398         from torch.utils.data import DataLoader
399         from detectors import build_detector
400         from trackers import build_tracker
401         from dataloaders import build_img_transforms, build_seq_transforms
402         from dataloaders.dataset_loader import ImageDataset
403         from utils import Center
404
405         streaming = frame_loader is not None
406
407         cfg = _build_cfg(weights, sport, device)
408         frames_in = int(cfg["model"]["frames_in"])
409         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
410         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
411
412         if streaming:
413             # `frames` is already the ordered list of (synthetic) frame paths.
414             if not isinstance(frames, (list, tuple)):
415                 raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
416             frames = list(frames)
417             frames_dir = source_key or "stream"
418         else:
419             frames_dir = frames
420             frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
glob.glob(osp.join(frames_dir, "*.jpg")))
421         if limit:
422             frames = frames[:limit]
423         if len(frames) < frames_in:
424             where = frames_dir if not streaming else "video stream"
425             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
426
427         # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
428         samples = []
429         for window in build_windows(frames, frames_in):
430             annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
431             samples.append({"frames": window, "annos": annos})
432         windows_total = len(samples)
433
434         # ---- cache setup / resume decision ---------------------------------- #
435         if cache_dir is None:
436             cache_dir = default_cache_dir(out_csv)
437         os.makedirs(cache_dir, exist_ok=True)
438         det_jsonl, _ = _cache_paths(cache_dir)
439         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
440         manifest = None if fresh else read_manifest(cache_dir)
441         resume = manifest_compatible(
442             manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
443             frames_in=frames_in, frames_total=len(frames), device=device,
```

```
        )
        if resume:
            windows_done, det_by_fid = load_and_clean_cache(cache_dir)
            logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
        else:
            if manifest is not None:
                logger.info("cache incompatible with this run -> starting fresh")
            reset_cache(cache_dir)
            windows_done, det_by_fid = 0, defaultdict(list)

        base_manifest = {
            "version": CACHE_VERSION,
            "frames_dir": cache_key,
            "weights": weights,
            "sport": sport,
            "frames_in": frames_in,
            "frames_total": len(frames),
            "device": device,
            "windows_total": windows_total,
            "windows_done": windows_done,
        }
        write_manifest(cache_dir, base_manifest)

        # ---- detector pass over the un-cached windows ---------------------- #
        remaining = samples[windows_done:]
        if remaining:
            _, transform_test = build_img_transforms(cfg)
            try:
                _, seq_transform_test = build_seq_transforms(cfg)
            except Exception:
                seq_transform_test = None
            ds = ImageDataset(cfg, remaining, input_wh, output_wh,
                              transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
            # Streaming forces single-process loading: the in-memory frame store + the
            # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
            loader_workers = 0 if streaming else num_workers
            loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
            detector = build_detector(cfg)

            from contextlib import ExitStack

            t0 = time.time()
            done = windows_done
            win_idx = windows_done
            log_every = max(1, log_every_batches)
            flush_n = max(1, flush_every)
            pending = []
            logger.info(f"detecting on {len(remaining)} remaining windows "
                        f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
                        f"source={'video-stream' if streaming else 'frames-dir'})...")
            det_f = open(det_jsonl, "a")
            try:
                with ExitStack() as stack:
                    stack.enter_context(torch.no_grad())
                    # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
                    # in-memory decoded frame for that synthetic path; every other line of
                    # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
                    # The detector's first needed frame is `windows_done` (window i starts
```

```
at
                    # frame i), so prime the store there for a resume.
                    if streaming:
                        stack.enter_context(
                            _streaming_read_image_patch(frame_loader, len(frames),
                                                        window=frames_in,
start_index=windows_done))
                    for bi, batch in enumerate(loader):
                        imgs, _hms, trans = batch[0], batch[1], batch[2]
                        img_paths = [list(t) for t in batch[-1]]   # [frame_in][batch] ->
path
                        batch_results, _ = detector.run_tensor(imgs, trans)
                        nb = imgs.shape[0]
                        for ib in range(nb):
                            frames_payload = []
                            for ie in sorted(batch_results[ib].keys()):
                                p = img_paths[ie][ib]
                                fid = _frame_id(p)
                                plain = [_det_plain(d) for d in batch_results[ib][ie]]
                                frames_payload.append([fid, plain])
                                det_by_fid[fid].extend(plain)
                            pending.append({"w": win_idx, "f": frames_payload})
                            win_idx += 1
                        done += nb
                        if len(pending) >= flush_n or done >= windows_total:
                            _append_windows(det_f, pending)
                            pending = []
                            base_manifest["windows_done"] = done
                            write_manifest(cache_dir, base_manifest)
                        if (bi + 1) % log_every == 0 or done >= windows_total:
                            el = time.time() - t0
                            new_done = done - windows_done
                            rate = new_done / el if el > 0 else 0.0
                            eta = (windows_total - done) / rate if rate > 0 else 0.0
                            logger.info(f"{done}/{windows_total} "
                                        f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
                                        f"elapsed {el:.0f}s | ETA {eta:.0f}s")
                    if pending:
                        _append_windows(det_f, pending)
                        base_manifest["windows_done"] = done
                        write_manifest(cache_dir, base_manifest)
            finally:
                det_f.close()
            logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
        else:
            logger.info("detector cache already complete -> tracker-only re-run")

        # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
        tracker = build_tracker(cfg)
        tracker.refresh()
        rows = []
        for fid in sorted(det_by_fid.keys()):
            r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
            rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))

        _atomic_write_trajectory(out_csv, rows)
        visible = sum(1 for _, v, *_ in rows if v)
        logger.info(f"wrote {len(rows)} rows -> {out_csv}")
        logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
        return out_csv


    def extract_frames(video_path: str, frames_dir: str) -> str:
        """Decode a video to PNG frames named by source frame index (00000000.png ...).
```

```

        Idempotent: if the dir already holds the expected number of frames (matching the
        container's reported frame count) we skip re-decoding ? re-extraction after a
        reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
        extracting with ffmpeg on the host and passing --frames_dir.
        """
        import cv2
        os.makedirs(frames_dir, exist_ok=True)
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        existing = len(glob.glob(osp.join(frames_dir, "*.png")))
        if expected > 0 and existing >= expected:
            cap.release()
            logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
            return frames_dir
        i = 0
        while True:
            ok, frame = cap.read()
            if not ok:
                break
            cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
            i += 1
        cap.release()
        logger.info(f"extracted {i} frames -> {frames_dir}")
        return frames_dir


    def _probe_frame_count(video_path: str) -> int:
        """Container-reported frame count via cv2 (0 if unknown). Used to size the
stream."""
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        cap.release()
        return n


    def make_video_frame_loader(video_path: str):
        """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.

        ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
        long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
        detector's sliding-window order); it reads sequentially and only uses a frame-seek
to
        skip forward when a *resume* starts past the current position. Each ``cap.read()``
        returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
would
        yield (PNG is lossless), so the detector sees identical pixels to the disk path.

        Returns the count hint from the container so the caller can build the frame list;
        ``release`` closes the capture.
        """
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        state = {"pos": 0}  # next index cap.read() will return

        def frame_loader(index: int):
            index = int(index)
```

```
            if index < state["pos"]:
                raise RuntimeError(
                    f"streaming decode is forward-only; got index {index} after
{state['pos']}")
            if index > state["pos"]:
                # Forward skip only happens when resuming past cached windows. Seek once.
                cap.set(cv2.CAP_PROP_POS_FRAMES, index)
                state["pos"] = index
            ok, frame = cap.read()
            if not ok:
                raise RuntimeError(f"failed to decode frame {index} from {video_path}")
            state["pos"] += 1
            return frame

        def release():
            cap.release()

        return frame_loader, count_hint, release


    def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
                            limit: int = 0, batch_size: int = 8,
                            log_every_batches: int = 50, cache_dir: "str | None" = None,
                            flush_every: int = 1000, fresh: bool = False,
                            device: str = "cuda") -> str:
        """Fast path: decode ``video_path`` on the fly and feed frames straight to the
        detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
        dominates a long clip). Output is bit-identical to the frames-on-disk path; see
        ``run``'s docstring for why.
        """
        frame_loader, count_hint, release = make_video_frame_loader(video_path)
        try:
            if count_hint <= 0:
                raise SystemExit(
                    f"could not determine frame count for {video_path}; cannot stream "
                    f"(fall back to frame extraction with --frames_dir)")
            frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
            source_key = "video:" + osp.abspath(video_path)
            return run(frame_list, weights, out_csv, sport, limit,
                       batch_size=batch_size, num_workers=0,
                       log_every_batches=log_every_batches, cache_dir=cache_dir,
                       flush_every=flush_every, fresh=fresh,
                       frame_loader=frame_loader, source_key=source_key, device=device)
        finally:
            release()


    def main():
        ap = argparse.ArgumentParser()
        ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
        ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
        ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
        ap.add_argument("--stream-video", action="store_true",
                        help="FAST PATH: decode --video on the fly and feed frames straight
to "
                             "the detector (no PNG extraction). Output-equivalent to the
disk path.")
        ap.add_argument("--weights", required=True)
        ap.add_argument("--out", required=True)
        ap.add_argument("--sport", default="badminton")
        ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
                        help="torch device for the detector (default cuda = the WSL parity
```

```
path)")
684         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
685         ap.add_argument("--batch-size", type=int, default=8)
686         ap.add_argument("--num-workers", type=int, default=4)
687         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
688         ap.add_argument("--cache-dir", default=None,
689                         help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
690         ap.add_argument("--flush-every", type=int, default=1000,
691                         help="fsync the detector cache every N windows (bounds reboot
loss)")
692         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
693         args = ap.parse_args()
694
695         if args.stream_video:
696             if not args.video:
697                 raise SystemExit("--stream-video requires --video")
698             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
699                                 batch_size=args.batch_size,
700                                 log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
701                                 flush_every=args.flush_every, fresh=args.fresh,
device=args.device)
702             return
703
704         frames_dir = args.frames_dir
705         if args.video:
706             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
707         if not frames_dir:
708             raise SystemExit("provide --frames_dir or --video")
709         run(frames_dir, args.weights, args.out, args.sport, args.limit,
710             batch_size=args.batch_size, num_workers=args.num_workers,
711             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
712             flush_every=args.flush_every, fresh=args.fresh, device=args.device)
713
714
715     if __name__ == "__main__":
716         main()
717
```

11. assistant (2026-06-20T08:57:28.545Z)

Now let me read the wasb_hybrid, fusion_hybrid, config models, and worker.

12. user (2026-06-20T08:57:30.083Z)

```
1       """WASB-based hybrid play segmenter.
2
3       Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4       WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5       badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7       For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8       candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9       min_window_duration) are what get tuned against golden rally labels.
10
11      The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12      lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13      WASB runner and identity.
14      """
15      from typing import Any, Dict, Tuple
```

```

    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
    from backend.pipeline.detectors.base import DetectorRunner
    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig


    class WasbHybridSegmenter(TrajectoryHybridSegmenter):
        family = "wasb"
        display_label = "WASB"

        @property
        def name(self) -> str:
            return "wasb_hybrid"

        @property
        def description(self) -> str:
            return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
                    "weights) for precise candidate rallies + AI provider for semantic
boundaries.")

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            cfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(cfg), {"weights_path": cfg.weights_path}

        def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
            # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
            # seam (imported only when detector_impl='native' is actually selected).
            from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
            cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
            return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
                                           "detector_impl": "native", "device": cfg.device}


    segmenter_registry.register(WasbHybridSegmenter)

```

13. user (2026-06-20T08:57:30.539Z)

```

    """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

    Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
    TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
    do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
    shuttle-seeded window via two overridable seams the engine exposes:

    - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
      GPU-free, computed from the trajectory we already have); and, only when the person cue
      is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
      (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
      fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
      torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
      model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
      via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
      (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
```

```
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32      from backend.pipeline.detectors.base import DetectorRunner
33      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35      from backend.pipeline.detectors import rally_gate
36      from backend.pipeline.detectors import fusion_features as ff
37
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
54          @property
55          def name(self) -> str:
56              return "fusion_hybrid"
57
58          @property
59          def description(self) -> str:
60              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                      "AI provider sets semantic boundaries.")
63
64          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66              if substrate == "tracknet":
67                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                  return TrackNetRunner(cfg), {
69                      "weights_path": cfg.weights_path,
```

```
                    "queue_length": cfg.queue_length,
                    "fusion_substrate": "tracknet",
                }
            wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}

        def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
            # Native (no-WSL) path supports the WASB substrate only ? there is no native
TrackNet
            # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
            substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
            if substrate != "wasb":
                raise NotImplementedError(
                    "detector_impl='native' supports the WASB substrate only ? set "
                    "indexing.fusion.substrate='wasb' (no native TrackNet runner yet).")
            from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
            cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
            return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
"detector_impl": "native",
                                           "fusion_substrate": "wasb", "device": cfg.device}

        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
            # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
            # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
            # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
            gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
                                            min_crossings=0,
estimate=self._axis_estimate(points))
            stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

            fcfg = idx_cfg.get("fusion", {})
            if fcfg.get("cue_flags", {}).get("person", False):
                stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
            return stats

        # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
        # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
        # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
        # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
        # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
        # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
        # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.

        def _axis_estimate(self, points: List[Any]) -> tuple:
            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
            computed once per video, not once per candidate window (it depends only on
```

```
points)."""
            key = id(points)
            if self._axis_cache_key != key or self._axis_cache is None:
                self._axis_cache_key = key
                self._axis_cache = rally_gate.estimate_play_axis(points)
            est = self._axis_cache
            assert est is not None
            return est

        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                             frame_width: float, video_path: str,
                             fcfg: Dict[str, Any]) -> Dict[str, float]:
            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
            frame range and compute the scale/translation-invariant person features. Lazily
builds
            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
            from backend.pipeline.detectors.person_detector import PersonDetector
            if self._person is None:
                self._person = PersonDetector(self.config)

            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
            fw = det["frame_width"] or frame_width
            fh = det["frame_height"]
            # If we can't trust the frame dims, every normalised feature would silently
collapse
            # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
            if fw <= 0 or fh <= 0:
                return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}

            poly = fcfg.get("court_polygon")
            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
            net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
            shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
            return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                              court_polygon=polygon, net=net)

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
            (0) keep every window ? identical to the substrate. Persisted candidates are
            untouched (full superset).

            NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
            thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
            where no segment clears ``min_crossings``), the gate falls back to the full
ungated
            set rather than emit a video with zero rallies. This makes Gate A/B strictly
            *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
            the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
```

```
167             below it always wins on an otherwise-empty video. Observed on real footage
168             (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
169             gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
170             fcfg = idx_cfg.get("fusion", {})
171             min_crossings = int(fcfg.get("min_crossings", 0))
172             min_players = int(fcfg.get("min_players", 0))
173             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
174
175             keep: List[int] = []
176             for i, st in enumerate(window_stats):
177                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
178                     continue
179                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
180                     continue
181                 keep.append(i)
182
183             if not keep and window_stats:
184                 # Gating emptied a non-empty video ? fall back to the ungated set.
185                 return list(range(len(window_stats)))
186             return keep
187
188
189     segmenter_registry.register(FusionSegmenter)
190
```

14. assistant (2026-06-20T08:57:33.172Z)

Now let me read the worker `__main__.py` and the config models.

15. user (2026-06-20T08:57:34.122Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
21      import os
22      import sys
23      import tempfile
24      from typing import Any, List
25
26      from backend import storage
27      from backend.config import load_config
28      from backend.infrastructure.database import Database
```

```
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.hashing import compute_video_id


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


    def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
                "status": status,
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
```

```
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI).
        val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(video_id, local_video, "failed" if failures else "processed",
                     [r.model_dump() for r in val_results])

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print("[worker] validation FAILED: "
                      + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
                return 2
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[worker] unknown segmenter: {seg_name!r} "
                      f"(have {list(segmenter_registry.list_available())})")
                return 2
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return 3
            segments = result.value if hasattr(result, "value") else result
            status = "success"
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db, video_id=video_id, video_path=local_video, config=config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=False, segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
```

```
148
149         out_prefix = args.out_uri.rstrip("/")
150         pushed = []
151         for fn in sorted(os.listdir(out_local)):
152             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
154             pushed.append(uri)
155
156         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
157         for u in pushed:
158             print("   ", u)
159         return 0
160
161
162     def _resolve_train_detector(args: argparse.Namespace, config: Any):
163         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
164         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
165         stub=CPU mock). Returns the runner, or prints an error + returns None.
166
167         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
168         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
169         no shuttle detector and is rejected.
170         """
171         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
172
173         seg_cls = segmenter_registry.get(args.segmenter)
174         if not seg_cls:
175             print(f"[worker] unknown segmenter: {args.segmenter!r} "
176                   f"(have {list(segmenter_registry.list_available())})")
177             return None
178         seg = seg_cls(None, config)
179         if not isinstance(seg, TrajectoryHybridSegmenter):
180             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
181                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
182             return None
183         runner, _ = seg._resolve_runner(config.get("indexing", {}))
184         ok, msg = runner.healthcheck()
185         if not ok:
186             print(f"[worker] detector environment not ready: {msg}")
187             return None
188         print(f"[worker] detector ready ({args.segmenter}): {msg}")
189         return runner
190
191
192     def cmd_train(args: argparse.Namespace) -> int:
193         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
194         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
195
196         Two trajectory sources (the train pipeline + harness are identical either way):
197           * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
198             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
199           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
200             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
201             is the M3.5 "first real training" path; only the trajectory source flips.
202         """
```

```
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
                            if u.lower().endswith(".csv"))
        if len(label_uris) < 2:
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
            name = fname.replace(".rallies.csv", "").replace(".csv", "")
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping (synth fixes them
at 30/1280).
                from backend.pipeline.segmenters.trajectory_hybrid import
TrajectoryHybridSegmenter
                fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
```

```
                                "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(prog="python -m backend.worker",
                                    description="Storage-aware worker: pull ? run pipeline ?
push.")
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
        pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
        pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
        pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
        pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
```

```
dir)")
313         pi.add_argument("--max-frames", type=int, default=None)
314         pi.add_argument("--set", action="append", default=[], metavar="k=v",
315                         help="config override, e.g. --set indexing.skip_ai_handoff=true")
316         pi.set_defaults(func=cmd_infer)
317
318         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
319         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
320                         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
321         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
322         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
323         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
324                         help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
325                              "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
326                              "(needs videos/ in the corpus; honours detector_impl).")
327         pt.add_argument("--segmenter", default="wasb_hybrid",
328                         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
329         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
330         pt.add_argument("--config", default=None)
331         pt.add_argument("--scratch", default=None)
332         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
333         pt.add_argument("--frame-width", type=float, default=1280.0,
334                         help="synth trajectory frame width (detector probes the video)")
335         pt.add_argument("--set", action="append", default=[], metavar="k=v")
336         pt.set_defaults(func=cmd_train)
337         return p
338
339
340     def main(argv=None) -> int:
341         args = build_parser().parse_args(argv)
342         return args.func(args)
343
344
345     if __name__ == "__main__":
346         sys.exit(main())
347
```

16. user (2026-06-20T08:57:34.464Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
```

```
    class ValidationRelevanceConfig(BaseModel):
        court_green_lower: list[int] = Field(default=[35, 40, 40])
        court_green_upper: list[int] = Field(default=[85, 255, 255])
        court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


    class ValidationConfig(BaseModel):
        min_resolution_width: int = 1280
        min_resolution_height: int = 720
        min_fps: float = 29.9
        min_blur_threshold: float = 20.0
        max_scene_cuts_per_minute: float = 0.5
        relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)


    class TrackNetConfig(BaseModel):
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/TrackNetV4"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "TrackNetV4"
        weights_path: str = ""
        queue_length: int = 5
        stage_in_wsl: bool = True
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
        # decoded frame cache) are deleted automatically ? they are pure intermediates,
        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
        # 0 = disabled.
        min_free_gb: float = 0.0


    class WasbIndexConfig(BaseModel):
        """Typed config for the live WASB shuttle substrate (indexing.wasb).

        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
        these knobs actually take effect ? previously this whole block was silently dropped
        because nested config sections defaulted to extra='ignore'.
        """
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/WASB-SBDT"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "wasb"
        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
        # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
        device: str = "cuda"        # torch device for the native runner: cuda | mps | cpu
        python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
        # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
        # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
        # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
```

```
77          keep_frames: bool = False
78          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
79          min_free_gb: float = 0.0
80          # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
81          # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
82          # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
83          # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
84          # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
85          stream_video: bool = False
86
87
88      class FusionConfig(BaseModel):
89          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
90
91          Every default reproduces control behaviour: the fusion segmenter persists the
92          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
93          true ? the person-cue features, but it does NOT gate the served handoff unless a
94          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
95          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
96          supplied ? off-court persons (spectators / adjacent courts) are excluded.
97          """
98          # Which trajectory substrate seeds the windows (shuttle stays primary).
99          substrate: Literal["wasb", "tracknet"] = "wasb"
100         # Windowing velocity threshold for the fusion family (the engine reads
101         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
102         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
103         velocity_threshold: float = 0.02
104         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
105         # enabled ? so the default path never imports torch / touches the GPU.
106         cue_flags: dict[str, bool] = Field(default_factory=dict)
107         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
108         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
109         min_crossings: int = Field(default=0, ge=0)
110         # Served Gate B: when the person cue is on, drop windows with median in-court
players
111         # < this. 0 = OFF.
112         min_players: int = Field(default=0, ge=0)
113         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
114         court_polygon: Optional[list[list[float]]] = None
115         # Net line for the person two-sided-motion split (person features only ? the shuttle
116         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
117         net_axis: Literal["x", "y"] = "y"
118         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
119
120
121     class IndexingConfig(BaseModel):
122         default_segmenter: str = "yolo_hybrid"
123         use_gpu: bool = True
124         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
125         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
126         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
127         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
128         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
129         detector_impl: str = "auto"
```

```
130         motion_threshold: float = 0.08
131         min_segment_duration: float = 3.0
132         dead_time_merge_gap: float = 2.0
133         chunk_duration: float = 90.0
134         chunk_overlap: float = 10.0
135         detector_model: str = "fasterrcnn_resnet50_fpn"
136         detector_score_threshold: float = 0.5
137         yolo_velocity_threshold: float = 0.15
138         yolo_frame_skip: int = 3
139         yolo_tracker: str = "bytetrack.yaml"
140         yolo_prefetch_workers: int = 2
141         min_action_window_duration: float = 2.0
142         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
143         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
144         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
145         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
146         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
147         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
148         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
149         max_window_duration: float = 6.0
150         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
151         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
152         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
153         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
154         # Only cuts (recall can't drop). OFF by default until measured/promoted.
155         window_hard_cap: bool = False
156         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
157         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
158         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
159         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
160         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
161         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
162         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
163         window_inactivity_end: float = 1.5
164         inactivity_rally_thresh: float = 0.20
165         inactivity_min_density: float = 0.30
166         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
167         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
168         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
169         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
170         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
171         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
```

```
window_hard_cap
172         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
173         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
174         window_blob_fallback: bool = False
175         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
176         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
177         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
178         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
179         window_blob_factor: float = 4.0
180         log_yolo_candidates: bool = True
181         store_yolo_candidates: bool = True
182         ai_handoff_padding: float = 2.0
183         ai_max_workers: int = 5
184         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
185         # chunks of a high-bitrate (4K) source blow past the provider upload cap
186         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
187         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
188         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
189         # Matches gemini_refine's --clip-scale-width default.
190         ai_chunk_scale_width: int = 1280
191         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
192         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
193         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
194         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
195         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
196         skip_ai_handoff: bool = False
197         # Optional debug knob: trim every video to the first N seconds before processing.
198         debug_duration: Optional[float] = None
199
200         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
201         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
202         fusion: FusionConfig = Field(default_factory=FusionConfig)
203
204         @field_validator("default_segmenter")
205         @classmethod
206         def segmenter_must_be_registered(cls, v: str) -> str:
207             # Registry check happens at runtime in main/segmenter selection.
208             # Here we just allow any string for forward compatibility.
209             return v
210
211         @model_validator(mode="after")
212         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
213             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
214             # step makes `while t < duration: t += step` loop forever, growing memory before
any
215             # GPU work. Reject the bad config at load time instead.
216             if self.chunk_overlap >= self.chunk_duration:
217                 raise ValueError(
218                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
219                     f"({self.chunk_duration}); otherwise chunking never advances."
220                 )
221             return self
222
223
```

```
    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
        # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
        # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
        # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
        # local detector over-fires and its false positives are mostly SHORT (motion blips,
        # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
        # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
        min_rally_duration: float = Field(default=0.0, ge=0.0)
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
```

```
281         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
282         # contain upload_dir or /api/process will reject the uploaded paths.
283         upload_dir: str = "output/uploads"
284         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
285         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
286         max_upload_mb: int = 4096
287         max_part_mb: int = 95
288         allowed_upload_extensions: List[str] = ["mp4", "mov"]
289
290
291     class StorageConfig(BaseModel):
292         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
293         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
294         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = Google
Drive
295         (follow-up). Per-backend settings are loose dicts passed to the backend constructor
?
296         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
297         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
298         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
299         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
300         output_root: str = ""
301         local: dict[str, Any] = Field(default_factory=dict)
302         s3: dict[str, Any] = Field(default_factory=dict)
303         gcs: dict[str, Any] = Field(default_factory=dict)
304         gdrive: dict[str, Any] = Field(default_factory=dict)
305
306
307     class AppConfig(BaseModel):
308         """Root configuration object.
309
310         All runtime configuration should be accessed via an instance of this class
311         (or the loader) rather than raw dict lookups.
312         """
313
314         sport: Sport = "badminton"
315         ai_provider: str = "gemini"
316         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
317         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
318         ai_model: str = "gemini-flash-latest"
319
320         validation: ValidationConfig = Field(default_factory=ValidationConfig)
321         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
322         output: OutputConfig = Field(default_factory=OutputConfig)
323         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
324         security: SecurityConfig = Field(default_factory=SecurityConfig)
325         logging: LoggingConfig = Field(default_factory=LoggingConfig)
326         storage: StorageConfig = Field(default_factory=StorageConfig)
327
328         # Allow extra keys during transition so existing config.json files do not break.
329         model_config = {
330             "extra": "allow",
331             "validate_assignment": True,
332         }
333
334         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
335         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
336         # attribute access / the typed config. A nested *section* is returned as a plain
dict
```

```
337         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
338         # section model instead breaks it (a model has no .get), which bit every validator
on
339         # real runs. Scalar leaves are returned as-is.
340         def get(self, key: str, default: Any = None) -> Any:
341             _MISSING = object()
342             val: Any = _MISSING
343             if key in type(self).model_fields or hasattr(self, key):
344                 val = getattr(self, key, _MISSING)
345             if val is _MISSING:
346                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
347                 # discovered dynamically from the model fields, so a future top-level
section
348                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
349                 for fname in type(self).model_fields:
350                     section = getattr(self, fname, None)
351                     if isinstance(section, BaseModel) and hasattr(section, key):
352                         val = getattr(section, key)
353                         break
354             if val is _MISSING:
355                 return default
356             if isinstance(val, BaseModel):
357                 return val.model_dump(mode="json")
358             return val
359
360         def __getitem__(self, key: str) -> Any:
361             val = self.get(key)
362             if val is None and key not in type(self).model_fields:
363                 # Re-raise like a real dict for unknown top-level
364                 raise KeyError(key)
365             return val
366
367
368     # Convenience type alias for callers that want to annotate
369     Config = AppConfig
370
```

17. assistant (2026-06-20T08:57:37.930Z)

Now let me read the new tests and check the git diff to confirm exactly what changed.

18. user (2026-06-20T08:57:38.888Z)

```
1       """NativeWasbRunner coverage ? the Linux-native (no-WSL) WASB detector path (M3.5).
2
3       The native subprocess is mocked (no torch/conda/GPU here). These tests lock the contract
4       the GPU-box byte-parity gate then relies on: it IS a DetectorRunner, it is NOT a WSL
runner
5       (no WslCommandMixin / wsl invocation), it builds a native argv (no `wsl`, no `conda
6       activate`, no `/mnt` translation) that runs the SAME wasb_infer.py with the SAME output
CSV
7       name as the WSL runner, threads the device through, and cleans the frame cache on
success.
8       """
9       import os
10      import types
11      from unittest.mock import patch
12
13      import pytest
14
15      from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
16      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin
```

```
17
18
19      def _proc(rc=0, stdout="", stderr=""):
20          return types.SimpleNamespace(returncode=rc, stdout=stdout, stderr=stderr)
21
22
23      # --- identity / no-WSL guarantee
------------------------------------------------------
24      def test_is_a_detector_runner():
25          assert isinstance(NativeWasbRunner(NativeWasbConfig()), DetectorRunner)
26
27
28      def test_is_not_a_wsl_runner():
29          """The whole point of M3.5: the native runner must carry NO WSL plumbing."""
30          r = NativeWasbRunner(NativeWasbConfig())
31          assert not isinstance(r, WslCommandMixin)
32          assert not hasattr(r, "_wsl_bash")
33
34
35      def test_reuses_model_agnostic_windowing():
36          from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
37          assert NativeWasbRunner.parse_trajectory_csv is TrackNetRunner.parse_trajectory_csv
38          assert NativeWasbRunner.trajectory_to_action_windows is
TrackNetRunner.trajectory_to_action_windows
39
40
41      # --- config resolution (env overrides win over config over default)
---------------------
42      def test_from_indexing_cfg_defaults():
43          cfg = NativeWasbConfig.from_indexing_cfg({})
44          assert cfg.device == "cuda" and cfg.python_bin == "python"
45          assert cfg.weights_path == "" and cfg.repo_dir == "~/models/WASB-SBDT"
46          assert cfg.stream_video is False and cfg.keep_frames is False
47
48
49      def test_from_indexing_cfg_reads_config_block():
50          cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
51              "weights_path": "/m/w.pth.tar", "device": "cpu", "python_bin":
"/envs/wasb/bin/python",
52              "repo_dir": "/m/WASB-SBDT", "stream_video": True, "keep_frames": True, "sport":
"tennis"}})
53          assert cfg.weights_path == "/m/w.pth.tar" and cfg.device == "cpu"
54          assert cfg.python_bin == "/envs/wasb/bin/python" and cfg.repo_dir == "/m/WASB-SBDT"
55          assert cfg.stream_video is True and cfg.keep_frames is True and cfg.sport ==
"tennis"
56
57
58      def test_env_overrides_beat_config(monkeypatch):
59          monkeypatch.setenv("WASB_WEIGHTS", "/env/w.pth.tar")
60          monkeypatch.setenv("WASB_DEVICE", "mps")
61          monkeypatch.setenv("WASB_PYTHON", "/env/py")
62          monkeypatch.setenv("WASB_REPO_DIR", "/env/repo")
63          monkeypatch.setenv("WASB_SCRATCH", "/env/scratch")
64          cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
65              "weights_path": "/cfg/w", "device": "cpu", "python_bin": "/cfg/py", "repo_dir":
"/cfg/repo"}})
66          assert cfg.weights_path == "/env/w.pth.tar" and cfg.device == "mps"
67          assert cfg.python_bin == "/env/py" and cfg.repo_dir == "/env/repo"
68          assert cfg.scratch_dir == "/env/scratch"
69
70
71      # --- healthcheck
------------------------------------------------------------------------
72      def _ok_cfg(tmp_path, **kw):
73          w = tmp_path / "w.pth.tar"
```

```
        w.write_text("weights")
        (tmp_path / "repo" / "src").mkdir(parents=True)
        return NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "repo"), **kw)


    def test_healthcheck_ok(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
        with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
True")):
            ok, msg = r.healthcheck()
        assert ok and "torch" in msg


    def test_healthcheck_empty_weights():
        ok, msg = NativeWasbRunner(NativeWasbConfig(weights_path="")).healthcheck()
        assert not ok and "weights" in msg.lower()


    def test_healthcheck_weights_missing(tmp_path):
        (tmp_path / "repo" / "src").mkdir(parents=True)
        cfg = NativeWasbConfig(weights_path=str(tmp_path / "nope.pth"),
repo_dir=str(tmp_path / "repo"))
        ok, msg = NativeWasbRunner(cfg).healthcheck()
        assert not ok and "not found" in msg.lower()


    def test_healthcheck_repo_missing(tmp_path):
        w = tmp_path / "w.pth.tar"
        w.write_text("x")
        cfg = NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "absent"))
        ok, msg = NativeWasbRunner(cfg).healthcheck()
        assert not ok and "repo" in msg.lower()


    def test_healthcheck_torch_import_fails(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path))
        with patch.object(r, "_run", return_value=_proc(1, stderr="ModuleNotFoundError:
torch")):
            ok, msg = r.healthcheck()
        assert not ok and "failed" in msg.lower()


    def test_healthcheck_cuda_unavailable_when_device_cuda(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
        with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
            ok, msg = r.healthcheck()
        assert not ok and "cuda" in msg.lower()


    def test_healthcheck_cpu_device_ok_without_cuda(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="cpu"))
        with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
            ok, msg = r.healthcheck()
        assert ok  # cpu run does not require a GPU


    def test_healthcheck_python_not_found(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, python_bin="/no/such/python"))
        with patch.object(r, "_run", side_effect=FileNotFoundError()):
            ok, msg = r.healthcheck()
        assert not ok and "python" in msg.lower()


```

```
    # --- run_predict
-----------------------------------------------------------------------
    def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
        """Drive run_predict to success with the subprocess mocked; returns (runner, out,
rm_spy, argv, cwd)."""
        r = NativeWasbRunner(cfg)
        key = f"clip__{vid[:12]}"
        (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n")  # success = CSV
exists
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", return_value=_proc(0)) as run_spy, \
             patch.object(r, "_rm_rf") as rm_spy:
            out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
        argv = run_spy.call_args[0][0]
        cwd = run_spy.call_args.kwargs.get("cwd")
        return r, out, rm_spy, argv, cwd, key


    def test_run_predict_builds_native_argv_no_wsl(tmp_path):
        _, out, rm, argv, cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path)
        assert out == os.path.join(os.path.abspath(str(tmp_path)), f"{key}_wasb.csv")
        # No WSL / conda plumbing anywhere in the command.
        assert "wsl" not in argv
        assert not any("conda" in a or "/mnt/" in a for a in argv)
        # It runs the SAME inference core, device-threaded, writing the SAME CSV name as
WSL.
        assert argv[0] == "python" and argv[1] == "wasb_infer.py"
        assert "--device" in argv and argv[argv.index("--device") + 1] == "cuda"
        assert "--weights" in argv and argv[argv.index("--weights") + 1] == "/m/w.pth"
        assert argv[argv.index("--out") + 1] == out
        # Disk mode extracts frames + cleans them on success.
        assert "--frames_out_dir" in argv and "--stream-video" not in argv
        assert cwd.endswith(os.path.join("src"))
        rm.assert_called_once()


    def test_run_predict_stream_mode_no_frames_dir(tmp_path):
        _, out, rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", stream_video=True), tmp_path)
        assert out is not None
        assert "--stream-video" in argv and "--frames_out_dir" not in argv
        rm.assert_not_called()  # streaming never materializes a frame cache


    def test_run_predict_device_threaded_cpu(tmp_path):
        _, _out, _rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", device="cpu"), tmp_path)
        assert argv[argv.index("--device") + 1] == "cpu"


    def test_run_predict_keep_frames_skips_cleanup(tmp_path):
        _, _out, rm, _argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", keep_frames=True), tmp_path)
        rm.assert_not_called()


    def test_run_predict_none_when_copy_infer_fails(tmp_path):
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=False):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_run_predict_none_when_subprocess_fails(tmp_path):
```

```
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", return_value=_proc(1, stderr="CUDA error")):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_run_predict_none_when_csv_absent(tmp_path):
        # Subprocess "succeeds" (rc 0) but writes no CSV -> treated as failure.
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", return_value=_proc(0)):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_run_predict_times_out_returns_none(tmp_path):
        import subprocess
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", side_effect=subprocess.TimeoutExpired(cmd="x",
timeout=1)):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_csv_name_matches_wsl_contract(tmp_path):
        """The native CSV name must equal the WSL runner's so downstream + the parity diff
line up."""
        _, out, _rm, _argv, _cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth"), tmp_path)
        assert os.path.basename(out) == f"{key}_wasb.csv"


    # --- the detector_impl='native' seam (parallels the stub seam)
-------------------------
    def test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
        runner, params = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl":
"native"})
        assert isinstance(runner, NativeWasbRunner)
        assert params.get("detector_impl") == "native"


    def test_resolve_runner_native_env_overrides_config(monkeypatch):
        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
        from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
        runner, _ = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl": "auto"})
        assert isinstance(runner, NativeWasbRunner)


    def test_resolve_runner_native_supported_for_fusion_wasb(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
        runner, params = FusionSegmenter(None, {})._resolve_runner({"detector_impl":
"native"})
        assert isinstance(runner, NativeWasbRunner)
        assert params.get("fusion_substrate") == "wasb"


    def test_resolve_runner_native_refused_for_fusion_tracknet(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
        with pytest.raises(NotImplementedError):
```

```
249             FusionSegmenter(None, {})._resolve_runner(
250                 {"detector_impl": "native", "fusion": {"substrate": "tracknet"}})
251
252
253     def test_resolve_runner_native_refused_for_tracknet(monkeypatch):
254         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
255         from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
256         with pytest.raises(NotImplementedError):
257             TrackNetHybridSegmenter(None, {})._resolve_runner({"detector_impl": "native"})
258
```

19. user (2026-06-20T08:57:39.878Z)

```
warning: in the working copy of 'backend/pipeline/segmenters/wasb_hybrid.py', LF will be
replaced by CRLF the next time Git touches it
 backend/config/models.py                          |   9 +-
 backend/pipeline/detectors/wasb_infer.py          |  43 ++++++---
 backend/pipeline/segmenters/fusion_hybrid.py      |  13 +++
 backend/pipeline/segmenters/trajectory_hybrid.py  |  28 ++++--
 backend/pipeline/segmenters/wasb_hybrid.py        |   8 ++
 backend/worker/__main__.py                        | 106 +++++++++++++++++++---
 tests/test_worker.py                              |  51 ++++++++++
 7 files changed, 222 insertions(+), 36 deletions(-)
---BRANCH---
feat/native-wasb-runner
```

20. assistant (2026-06-20T08:57:43.985Z)

```
The diff stat doesn't list `native_wasb_runner.py` or `test_native_wasb_runner.py` ? those are
untracked (new files). Let me verify they show as untracked, and check `_resolve_runner` for the
`idx_cfg.get` call on a dict vs config object. Let me also examine the worker's
`_resolve_train_detector` more closely, since it calls `config.get("indexing", {})`.
```

21. user (2026-06-20T08:57:45.107Z)

```
 M backend/config/models.py
 M backend/pipeline/detectors/wasb_infer.py
 M backend/pipeline/segmenters/fusion_hybrid.py
 M backend/pipeline/segmenters/trajectory_hybrid.py
 M backend/pipeline/segmenters/wasb_hybrid.py
 M backend/worker/__main__.py
 M tests/test_worker.py
?? artifacts/
?? backend/pipeline/detectors/native_wasb_runner.py
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
?? tests/test_native_wasb_runner.py
---
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index 9b1f202..f7edb9d 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -51,17 +51,27 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
     display_label: str = ""

     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
-        """Return (runner, detector-specific detection_params extras)."""
+        """Return (runner, detector-specific detection_params extras) for the default (WSL)
path."""
         raise NotImplementedError

+    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
+        """Return (runner, extras) for the Linux-native (no-WSL) path. Only families with a
```

```
+        native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
+        raise NotImplementedError(
+            f"detector_impl='native' is not supported for the "
+            f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB has a "
+            f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
+
     def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
-        """Resolve the detector runner, honouring the CPU-stub override.
+        """Resolve the detector runner, honouring the CPU-stub and Linux-native overrides.

         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole pipeline
-        is regression-testable without a GPU ? "mock a GPU with a CPU"
-        (``docs/DECOUPLED_COMPUTE/``). Anything else delegates to ``_build_runner`` (the
-        real WSL/native runner) so the default production path is unchanged.
+        is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"`` swaps
+        in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
+        ``_build_native_runner``. Anything else (``"auto"``) delegates to ``_build_runner``
+        (the WSL runner) so the default production path is unchanged
+        (``docs/DECOUPLED_COMPUTE/``).
         """
         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
         if impl == "stub":
@@ -69,6 +79,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
                         self.display_label or "trajectory")
             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {"detector_impl":
"stub"}
+        if impl in ("native", "native_wasb", "wasb_native"):
+            logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
+                        self.display_label or "trajectory")
+            return self._build_native_runner(idx_cfg)
         return self._build_runner(idx_cfg)

     @staticmethod
@@ -206,9 +220,9 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
         ok, msg = runner.healthcheck()
         if not ok:
-            logger.error("%s WSL environment not ready: %s", label, msg)
+            logger.error("%s detector environment not ready: %s", label, msg)
             return []
-        logger.info("%s WSL env OK: %s", label, msg)
+        logger.info("%s detector env OK: %s", label, msg)

         # --- Phase 2: trajectory -> candidate rally windows ---
         # Trajectory detectors process the whole video in one pass (they carry
```

22. assistant (2026-06-20T08:57:51.185Z)

Now I have a full picture. Let me focus on the key concern: cross-platform path handling and the no-WSL guarantee. Let me trace a few critical things.

First, the no-WSL audit of `native_wasb_runner.py` ? confirmed it imports `DetectorRunner` and `TrackNetRunner` only (not `WslCommandMixin`, `to_wsl_mnt_path`, or `wsl_tilde_quote`). No `wsl`, `conda activate`, or `/mnt` in the argv construction. That part is clean.

Now let me dig into the cross-platform bugs. The critical issue is `run_predict`'s `os.path.abspath(str(video_win_path))` on line 163. Let me trace what happens with a real Linux path vs what the test passes. Let me also check the `_resolve_train_detector` path where `config.get("indexing", {})` returns a dict.

23. user (2026-06-20T08:57:56.287Z)

```
<string>:13: SyntaxWarning: invalid escape sequence '\c'
--- On LINUX (posixpath), a REAL linux video path ---
abspath: /home/ubuntu/videos/clip.mp4
basename of norm: clip.mp4
splitext: ('clip', '.mp4')

--- On LINUX (posixpath), a WINDOWS path passed by a TEST ---
repr: 'C:\x0b\\clip.mp4'
abspath (posix): C:\Users\avidu\Projects\badminton-highlight-indexer/C:
\clip.mp4
basename of norm: clip.mp4
```

24. assistant (2026-06-20T08:57:58.854Z)

The escape mangled it. Let me redo that test cleanly with a raw Windows path string to confirm the real behavior on Linux.

25. user (2026-06-20T08:58:04.523Z)

```
<stdin>:11: SyntaxWarning: invalid escape sequence '\c'
=== LINUX production: real Linux video path ===
os.path.abspath(video): /data/videos/clip.mp4

=== LINUX test machine running the WINDOWS-style path the test passes ===
raw: 'C:\x0b\\clip.mp4'
os.path.abspath(video) on linux: 'C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer/C:\x0b\\clip.mp4'
norm: C:
/clip.mp4
basename(norm): clip.mp4

=== But the argv --video uses os.path.abspath(str(video_win_path)) (NOT norm) ===
argv --video value (linux): 'C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer/C:\x0b\\clip.mp4'
```

26. assistant (2026-06-20T08:58:10.929Z)

The `\v` is being interpreted as a vertical tab even in the heredoc because Python parses it. The test uses `r"C:\v\clip.mp4"` (raw). The key insight holds regardless: `os.path.abspath` of a Windows-style path on Linux mangles it. But this only matters for the **test** path, not the production Linux path. On production Linux, `video_win_path` will be a real Linux path which `abspath` handles fine. The test mocks `_run`, so it never inspects the argv `--video` value for correctness of the path ? it only checks structural things. So this is benign for production but worth noting.

Let me now look at the more important real-Linux correctness concerns. Let me verify the cwd `_repo_src()` and the worker's `config.get("indexing", {})` returning a dict (since `AppConfig.get` returns `model_dump`). Let me confirm what type `_resolve_runner` receives in the worker train path.

27. user (2026-06-20T08:58:16.242Z)

```
type(idx): <class 'dict'>
detector_impl: auto
type(wasb): <class 'dict'>
device: cuda python_bin: python weights_path: ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
```

28. assistant (2026-06-20T08:58:23.290Z)

Important finding: the default `WasbIndexConfig.weights_path` is `~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar` (non-empty), but `NativeWasbConfig` default

`weights_path` is `""`. When the worker/segmenter builds the native config via `from_indexing_cfg`, the typed config supplies the WSL default weights path. Let me verify the native config picks that up and whether it's the correct WASB path. This is actually fine ? it inherits the WSL weights default. But note the env-first logic: `env("WASB_WEIGHTS") or w.get("weights_path", "") or ""`. Since `w.get("weights_path")` returns the typed default, that's fine.

Now let me check a subtle but real cross-platform concern: the `scratch_dir` resolution and `frames_dir` join, plus whether `_repo_src()` produces a correct path on Linux. Let me also verify the worker train path passes the right `idx_cfg` and check `test_worker.py`.

29. user (2026-06-20T08:58:24.031Z)

```
1       """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3       `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4       segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5       on the droplet against MinIO.
6       """
7       import json
8
9       import pytest
10
11      from backend.config import load_config
12      from backend.worker import __main__ as wmain
13
14
15      # ---------------------------------------------------------------- pure helpers
16
17      @pytest.mark.parametrize("raw,expected", [
18          ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid",
"wasb_hybrid"),
19      ])
20      def test_coerce(raw, expected):
21          assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)
22
23
24      def test_apply_overrides_on_typed_config():
25          cfg = _fresh_config()
26          wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
27                                       "sport=tennis"])
28          assert cfg.indexing.skip_ai_handoff is True
29          assert cfg.indexing.min_segment_duration == 1.5
30          assert cfg.sport == "tennis"
31
32
33      def test_apply_overrides_rejects_bad_format():
34          cfg = _fresh_config()
35          with pytest.raises(ValueError):
36              wmain._apply_overrides(cfg, ["no_equals_sign"])
37
38
39      def test_write_intervals_csv(tmp_path):
40          segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
41                  {"start_time": 9.0, "end_time": 12.5}]
42          out = tmp_path / "intervals.csv"
43          wmain._write_intervals_csv(segs, str(out))
44          lines = out.read_text().splitlines()
45          assert lines[0] == "start,end,shot_count,ending_reason"
46          assert lines[1].startswith("2.000,5.000,4,")
47          assert lines[2].startswith("9.000,12.500,,")
48
```

```
49
50      def test_write_report_json(tmp_path):
51          out = tmp_path / "report.json"
52          wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success",
str(out))
53          data = json.loads(out.read_text())
54          assert data["video_id"] == "vid1" and data["segment_count"] == 1
55          assert data["segments"][0] == {"start": 1.0, "end": 2.0}
56
57
58      def test_parser_infer_accepts_set_and_uris():
59          args = wmain.build_parser().parse_args([
60              "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
61              "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
62          assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
63          assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
64
65
66      # ------------------------------------------------------------- cmd_infer (local,
mocked)
67
68      class _OKResult:
69          passed = True
70          validator_name = "fake"
71          message = "ok"
72
73          def model_dump(self):
74              return {"validator_name": "fake", "passed": True, "message": "ok"}
75
76
77      class _FailResult(_OKResult):
78          passed = False
79          message = "too blurry"
80
81
82      class _FakeSeg:
83          def __init__(self, db, config):
84              pass
85
86          def process_video(self, video_id, path, max_frames=None):
87              return [
88                  {"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
89                  {"start_time": 9.0, "end_time": 12.0, "shot_count": 6,
"shot_count_confidence": "LocalNoAI"},
90              ]
91
92
93      def _fresh_config():
94          # Load defaults without touching any cwd config.json.
95          import tempfile
96          p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
97          p.write("{}")
98          p.close()
99          return load_config(p.name)
100
101
102     @pytest.fixture
103     def mocked_pipeline(monkeypatch):
104         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
105         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
106                             lambda path, cfg: ([_OKResult()], {}))
107         monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
108         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
109
```

```
110
111     def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
112         video = tmp_path / "in.mp4"
113         video.write_bytes(b"FAKEVIDEO")
114         cfg = tmp_path / "config.json"
115         cfg.write_text("{}")
116         out = tmp_path / "bucket"
117         scratch = tmp_path / "scratch"
118
119         rc = wmain.main([
120             "infer", "--video-uri", str(video), "--out-uri", str(out),
121             "--config", str(cfg), "--scratch", str(scratch),
122             "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
123         ])
124         assert rc == 0
125         intervals = out / "outputs" / "vid123" / "intervals.csv"
126         report = out / "outputs" / "vid123" / "report.json"
127         assert intervals.exists() and report.exists()
128         rows = intervals.read_text().splitlines()
129         assert len(rows) == 3                          # header + 2 rallies
130         assert json.loads(report.read_text())["segment_count"] == 2
131
132
133     def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
134         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
135         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
136                             lambda path, cfg: ([_FailResult()], {}))
137         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
138         video = tmp_path / "in.mp4"
139         video.write_bytes(b"X")
140         cfg = tmp_path / "config.json"
141         cfg.write_text("{}")
142         rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path /
"o"),
143                          "--config", str(cfg), "--scratch", str(tmp_path / "s")])
144         assert rc == 2
145
146
147     # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---
148
149     def test_cmd_train_orchestration(tmp_path, monkeypatch):
150         import os
151         import subprocess
152
153         labels_dir = tmp_path / "corpus" / "labels"
154         labels_dir.mkdir(parents=True)
155         header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
156         (labels_dir / "v1.rallies.csv").write_text(header +
"1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
157         (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
158         out_dir = tmp_path / "out"
159         scratch = tmp_path / "scratch"
160         cfg = tmp_path / "config.json"
161         cfg.write_text("{}")
162
163         # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
164         def fake_run(cmd, *a, **k):
165             out = cmd[cmd.index("--out") + 1]
166             ver = cmd[cmd.index("--version") + 1]
167             bdir = os.path.join(out, ver)
168             os.makedirs(bdir, exist_ok=True)
169             for fn in ("weights.json", "manifest.json"):
170                 with open(os.path.join(bdir, fn), "w") as f:
171                     f.write("{}")
```

```
172             return subprocess.CompletedProcess(cmd, 0)
173         monkeypatch.setattr(subprocess, "run", fake_run)
174 
175         rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
str(out_dir),
176                          "--version", "0.0.1", "--config", str(cfg), "--scratch",
str(scratch)])
177         assert rc == 0
178         # bundle pushed to <out-uri>/weights/<version>/
179         assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
180         assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
181         # the manifest fed to the harness enumerated both labeled videos with a trajectory
each
182         specs = json.loads((scratch / "manifest.json").read_text())
183         assert len(specs) == 2
184         assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)
185 
186 
187     def test_parser_train_detector_args():
188         args = wmain.build_parser().parse_args([
189             "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version",
"1.0.0",
190             "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
191         assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"
192 
193 
194     def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
195         """--trajectory-source detector pulls videos + runs the resolved DetectorRunner.
With
196         RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the
runner)."""
197         import os
198         import subprocess
199 
200         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # resolve a CPU stub runner (no
GPU/WSL)
201         corpus = tmp_path / "corpus"
202         labels_dir = corpus / "labels"
203         videos_dir = corpus / "videos"
204         labels_dir.mkdir(parents=True)
205         videos_dir.mkdir(parents=True)
206         header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
207         (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
208         (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
209         (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDEO1")
210         (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDEO2")
211         out_dir = tmp_path / "out"
212         scratch = tmp_path / "scratch"
213         cfg = tmp_path / "config.json"
214         cfg.write_text("{}")
215 
216         def fake_run(cmd, *a, **k):
217             out = cmd[cmd.index("--out") + 1]
218             ver = cmd[cmd.index("--version") + 1]
219             bdir = os.path.join(out, ver)
220             os.makedirs(bdir, exist_ok=True)
221             for fn in ("weights.json", "manifest.json"):
222                 with open(os.path.join(bdir, fn), "w") as f:
223                     f.write("{}")
224             return subprocess.CompletedProcess(cmd, 0)
225         monkeypatch.setattr(subprocess, "run", fake_run)
226 
227         rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
228                          "--version", "0.0.2", "--config", str(cfg), "--scratch",
str(scratch),
```

```
229                          "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
230         assert rc == 0
231         specs = json.loads((scratch / "manifest.json").read_text())
232         assert len(specs) == 2
233         # Real-extract path: each spec points at an on-disk trajectory the runner actually
wrote.
234         assert all(os.path.exists(s["trajectory"]) for s in specs)
235         assert all(s["trajectory"].endswith("_stub.csv") for s in specs)  # the stub
runner's output
236
```

30. assistant (2026-06-20T08:58:26.623Z)

Now let me run the new tests and the broader suite to confirm green, plus run ruff/mypy on the new files per the pre-LGTM gate.

31. user (2026-06-20T08:58:35.994Z)

```
.........................................                                [100%]
43 passed in 5.99s
```

32. user (2026-06-20T08:58:44.159Z)

```
All checks passed!
=== MYPY ===
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 2 source files
```

33. assistant (2026-06-20T08:58:53.263Z)

Clean. Now let me focus on hunting the real cross-platform bugs. The key concern from the prompt is the `_resolve_train_detector` worker path. Let me look more carefully: in `cmd_train` detector source, the runner is built once and reused across videos. But `run_predict` writes to `traj_dir` which is the SAME directory for every video. The native runner uses `frames_dir = os.path.join(scratch, f"{key}_frames")` where `scratch = self.cfg.scratch_dir or output_dir`. With env `WASB_SCRATCH` unset, scratch falls back to `output_dir` = `traj_dir` (the abspath of it). That's fine.

But there's a subtler issue: the worker train passes `traj_dir` (relative path `scratch/trajectories`) to `run_predict`, which does `os.path.abspath`. Good.

Let me now examine a genuine concern: **`os.path.expanduser` of the default `repo_dir="~/models/WASB-SBDT"` on the GPU box, and how the `_copy_infer_script` interacts with the resumable cache keying**. Let me check the cache-key consistency issue between disk mode and what the native runner does. Let me also verify the `make_video_frame_loader` count check and the device-keyed manifest don't break parity.

The biggest correctness question per the acceptance gate is **byte-parity**. Let me compare the exact argv the WSL runner builds vs the native runner, paying attention to `--frames_out_dir` and the cache dir defaults.

34. user (2026-06-20T08:59:02.839Z)

```
device default parity: WSL omits --device (main default cuda); native passes --device cuda ->
SAME effective override
frames_out_dir handling OK on Linux
```

35. assistant (2026-06-20T08:59:06.766Z)

Now let me dig into a real parity concern I noticed: the **device-keyed manifest in `manifest_compatible`** and a potential parity break. The acceptance gate says "device default 'cuda' must keep wasb_infer's Hydra overrides identical to before (parity)." Let me check the `_build_cfg` change carefully ? the prior version. Let me look at the git diff for wasb_infer.py.

36. user (2026-06-20T08:59:07.474Z)

```
diff --git a/backend/pipeline/detectors/wasb_infer.py b/backend/pipeline/detectors/wasb_infer.py
index 7b589d5..5df5939 100644
--- a/backend/pipeline/detectors/wasb_infer.py
+++ b/backend/pipeline/detectors/wasb_infer.py
@@ -53,16 +53,22 @@ _DET_FILE = "det_raw.jsonl"
 _MANIFEST_FILE = "manifest.json"


-def _build_cfg(weights: str, sport: str):
-    """Compose the WASB eval config via Hydra, overriding model/weights/device."""
+def _build_cfg(weights: str, sport: str, device: str = "cuda"):
+    """Compose the WASB eval config via Hydra, overriding model/weights/device.
+
+    ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity preserved),
+    ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests ``runner.gpus=[0]``;
+    cpu/mps run with no GPU list so the detector places tensors on ``device``.
+    """
     from hydra import compose, initialize
+    gpus = "[0]" if device == "cuda" else "[]"   # single GPU on cuda; none for cpu/mps
     with initialize(config_path="configs", version_base=None):
         cfg = compose(config_name="eval", overrides=[
             f"dataset={sport}",
             "model=wasb",
             f"detector.model_path={weights}",
-            "runner.device=cuda",
-            "runner.gpus=[0]",      # this box has a single GPU; default config assumes more
+            f"runner.device={device}",
+            f"runner.gpus={gpus}",  # default config assumes multiple GPUs; pin to this box
         ])
     return cfg

@@ -129,8 +135,13 @@ def read_manifest(cache_dir: str):


 def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
-                        frames_in: int, frames_total: int) -> bool:
-    """A cache is reusable only if the run that produced it matches this run's inputs."""
+                        frames_in: int, frames_total: int, device: str = "cuda") -> bool:
+    """A cache is reusable only if the run that produced it matches this run's inputs.
+
+    ``device`` defaults to ``cuda`` so caches written before device-keying (no ``device``
+    field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda cache
+    (detections can differ numerically across devices), and vice-versa.
+    """
     if not manifest or manifest.get("version") != CACHE_VERSION:
         return False
     return (
@@ -139,6 +150,7 @@ def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str,
sport:
         and manifest.get("sport") == sport
         and manifest.get("frames_in") == frames_in
         and manifest.get("frames_total") == frames_total
+        and manifest.get("device", "cuda") == device
     )


@@ -360,7 +372,8 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",
```

```
         limit: int = 0, batch_size: int = 8, num_workers: int = 4,
         log_every_batches: int = 50, cache_dir: "str | None" = None,
         flush_every: int = 1000, fresh: bool = False,
-        frame_loader=None, source_key: "str | None" = None) -> str:
+        frame_loader=None, source_key: "str | None" = None,
+        device: str = "cuda") -> str:
     """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.

     Two equivalent ways to supply pixels (output is bit-identical between them):
@@ -391,7 +404,7 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",

     streaming = frame_loader is not None

-    cfg = _build_cfg(weights, sport)
+    cfg = _build_cfg(weights, sport, device)
     frames_in = int(cfg["model"]["frames_in"])
     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
@@ -427,7 +440,7 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",
     manifest = None if fresh else read_manifest(cache_dir)
     resume = manifest_compatible(
         manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
-        frames_in=frames_in, frames_total=len(frames),
+        frames_in=frames_in, frames_total=len(frames), device=device,
     )
     if resume:
         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
@@ -445,6 +458,7 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",
         "sport": sport,
         "frames_in": frames_in,
         "frames_total": len(frames),
+        "device": device,
         "windows_total": windows_total,
         "windows_done": windows_done,
     }
@@ -630,7 +644,8 @@ def make_video_frame_loader(video_path: str):
 def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str = "badminton",
                         limit: int = 0, batch_size: int = 8,
                         log_every_batches: int = 50, cache_dir: "str | None" = None,
-                        flush_every: int = 1000, fresh: bool = False) -> str:
+                        flush_every: int = 1000, fresh: bool = False,
+                        device: str = "cuda") -> str:
     """Fast path: decode ``video_path`` on the fly and feed frames straight to the
     detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
     dominates a long clip). Output is bit-identical to the frames-on-disk path; see
@@ -648,7 +663,7 @@ def run_video_streaming(video_path: str, weights: str, out_csv: str, sport:
 str
                     batch_size=batch_size, num_workers=0,
                     log_every_batches=log_every_batches, cache_dir=cache_dir,
                     flush_every=flush_every, fresh=fresh,
-                    frame_loader=frame_loader, source_key=source_key)
+                    frame_loader=frame_loader, source_key=source_key, device=device)
     finally:
         release()

@@ -664,6 +679,8 @@ def main():
     ap.add_argument("--weights", required=True)
     ap.add_argument("--out", required=True)
     ap.add_argument("--sport", default="badminton")
+    ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
+                    help="torch device for the detector (default cuda = the WSL parity path)")
     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick tests)")
     ap.add_argument("--batch-size", type=int, default=8)
     ap.add_argument("--num-workers", type=int, default=4)
@@ -681,7 +698,7 @@ def main():
```

```
         run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
                             batch_size=args.batch_size,
                             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
-                            flush_every=args.flush_every, fresh=args.fresh)
+                            flush_every=args.flush_every, fresh=args.fresh, device=args.device)
         return

     frames_dir = args.frames_dir
@@ -692,7 +709,7 @@ def main():
     run(frames_dir, args.weights, args.out, args.sport, args.limit,
         batch_size=args.batch_size, num_workers=args.num_workers,
         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
-        flush_every=args.flush_every, fresh=args.fresh)
+        flush_every=args.flush_every, fresh=args.fresh, device=args.device)


 if __name__ == "__main__":
```

37. assistant (2026-06-20T08:59:11.761Z)

The wasb_infer.py changes preserve cuda parity: `device="cuda"` default ? `gpus="[0]"` ? identical Hydra overrides as before. The manifest `device` key uses `.get("device", "cuda")` so old caches stay compatible. Good.

Now let me verify there are no existing tests for wasb_infer.py that might break with the device-keyed manifest, and run the full detector test suite.

38. user (2026-06-20T08:59:23.431Z)

```
.......................................................................... [ 39%]
.......................................................................... [ 78%]
.....................................                                    [100%]
184 passed, 807 deselected in 8.56s
```

39. assistant (2026-06-20T08:59:26.454Z)

Now let me run the full suite quickly to confirm nothing else regressed, then I'll finalize the cross-platform analysis.

40. user (2026-06-20T09:00:51.192Z)

```
 obsreport (optional reporting dep): available
============================================================================
.......................................................................... [  7%]
.......................................................................... [ 14%]
.......................................................................... [ 21%]
.......................................................................... [ 29%]
.......................................................................... [ 36%]
.......................................................................... [ 43%]
.......................................................................... [ 50%]
.......................................................................... [ 58%]
.......................................................................... [ 65%]
.......................................................................... [ 72%]
.......................................................................... [ 79%]
.......................................................................... [ 87%]
.......................................................................... [ 94%]
.....................................................                    [100%]
991 passed in 81.06s (0:01:21)
============================================================================
 RESULT: PASS - all tests green
============================================================================
```

41. assistant (2026-06-20T09:00:56.481Z)

Full suite is green (991 passed). Now let me complete the targeted cross-platform bug hunt. Let me re-examine the specific items the prompt flagged, focusing on real Linux production correctness.

The one genuine concern I want to verify is the **frame cache vs resumable detector cache** interaction in the native runner. On a failure, the native runner returns without cleaning `frames_dir`, but the resumable detector cache (`<out>_wasbcache/`) lives next to the CSV in `output_dir`. On success, it cleans `frames_dir` but NOT the `_wasbcache` dir. Let me check whether the WSL runner cleans the wasbcache ? if there's an asymmetry that leaks disk on the native path.

42. user (2026-06-20T09:00:58.073Z)

```
backend\pipeline\detectors\native_wasb_runner.py:110:    def _rm_rf(self, path: Optional[str])
-> None:
backend\pipeline\detectors\native_wasb_runner.py:201:            self._rm_rf(frames_dir)
backend\pipeline\detectors\base.py:129:    def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
backend\pipeline\detectors\wasb_infer.py:31:The cache is keyed by the output path
(``<out>_wasbcache/`` by default), lives next
backend\pipeline\detectors\wasb_infer.py:86:def _cache_paths(cache_dir: str):
backend\pipeline\detectors\wasb_infer.py:87:    return osp.join(cache_dir, _DET_FILE),
osp.join(cache_dir, _MANIFEST_FILE)
backend\pipeline\detectors\wasb_infer.py:90:def default_cache_dir(out_csv: str) -> str:
backend\pipeline\detectors\wasb_infer.py:94:    return osp.join(d, stem + "_wasbcache")
backend\pipeline\detectors\wasb_infer.py:121:def write_manifest(cache_dir: str, manifest: dict)
-> None:
backend\pipeline\detectors\wasb_infer.py:122:    _, mpath = _cache_paths(cache_dir)
backend\pipeline\detectors\wasb_infer.py:126:def read_manifest(cache_dir: str):
backend\pipeline\detectors\wasb_infer.py:127:    _, mpath = _cache_paths(cache_dir)
backend\pipeline\detectors\wasb_infer.py:157:def reset_cache(cache_dir: str) -> None:
backend\pipeline\detectors\wasb_infer.py:159:    det_jsonl, mpath = _cache_paths(cache_dir)
backend\pipeline\detectors\wasb_infer.py:165:def load_and_clean_cache(cache_dir: str):
backend\pipeline\detectors\wasb_infer.py:174:    det_jsonl, _ = _cache_paths(cache_dir)
backend\pipeline\detectors\wasb_infer.py:373:        log_every_batches: int = 50, cache_dir:
"str | None" = None,
backend\pipeline\detectors\wasb_infer.py:435:    if cache_dir is None:
backend\pipeline\detectors\wasb_infer.py:436:        cache_dir = default_cache_dir(out_csv)
backend\pipeline\detectors\wasb_infer.py:437:    os.makedirs(cache_dir, exist_ok=True)
backend\pipeline\detectors\wasb_infer.py:438:    det_jsonl, _ = _cache_paths(cache_dir)
backend\pipeline\detectors\wasb_infer.py:440:    manifest = None if fresh else
read_manifest(cache_dir)
backend\pipeline\detectors\wasb_infer.py:446:        windows_done, det_by_fid =
load_and_clean_cache(cache_dir)
backend\pipeline\detectors\wasb_infer.py:451:        reset_cache(cache_dir)
backend\pipeline\detectors\wasb_infer.py:465:    write_manifest(cache_dir, base_manifest)
backend\pipeline\detectors\wasb_infer.py:527:                        write_manifest(cache_dir,
base_manifest)
backend\pipeline\detectors\wasb_infer.py:539:                    write_manifest(cache_dir,
base_manifest)
backend\pipeline\detectors\wasb_infer.py:646:                        log_every_batches: int =
50, cache_dir: "str | None" = None,
backend\pipeline\detectors\wasb_infer.py:664:
log_every_batches=log_every_batches, cache_dir=cache_dir,
backend\pipeline\detectors\wasb_infer.py:689:                    help="resumable detector cache
dir (default: <out>_wasbcache next to --out)")
backend\pipeline\detectors\wasb_infer.py:700:
log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
backend\pipeline\detectors\wasb_infer.py:711:        log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
backend\pipeline\detectors\wasb_runner.py:194:            self._wsl_rm_rf(stale)
backend\pipeline\detectors\tracknet_runner.py:209:            self._wsl_rm_rf([wsl_video])
```

43. assistant (2026-06-20T09:01:10.259Z)

Confirmed: neither the WSL runner nor the native runner cleans the `<out>_wasbcache/` dir on

success ? both only clean the frame cache. So that's symmetric (pre-existing behavior, not a regression introduced here). Not a finding for this change.

Now, the critical cross-platform parity concern. The WSL runner writes frames to `~/clips/<key>_frames` (the WSL stage dir), and the resumable cache lands next to `--out` (the output dir). The native runner writes frames to `scratch/<key>_frames`. The `--out` CSV path is the same dir contract for both. The **cache key** in `manifest_compatible` for disk mode is `osp.abspath(frames_dir)` ? which differs between WSL (`~/clips/...`) and native (`scratch/...`). But since these are different machines, no clash. Parity of the OUTPUT CSV is what matters, and that's driven by pixels + weights + device, not the frames_dir path. Good.

Let me verify one last real bug candidate: the native runner's `_run` uses `cwd=self._repo_src()` and runs `python wasb_infer.py` (relative). The WSL runner does `cd repo/src && python wasb_infer.py`. wasb_infer's `_build_cfg` uses `initialize(config_path="configs", version_base=None)` ? a **relative** Hydra config path resolved against the **caller's file location** (wasb_infer.py is copied into `repo/src/`), not cwd. Let me confirm Hydra's `config_path` is relative to the .py file, which means cwd matters less but the script must be in `src/`. The native runner copies it there via `_copy_infer_script`. Good ? parity preserved.

Let me do a final check on a potential real Linux issue: `os.path.abspath(str(video_win_path))` when the worker passes a path. In the worker train path, `local_video` is `os.path.join(videos_dir, name)` ? a real local path. On Linux, `abspath` is correct. Fine.

Let me verify the one thing that could actually bite on Linux: whether `shutil.copy(_INFER_PY, ...)` and the `_INFER_PY` resolution work when the package is installed/run from the GPU box.

44. user (2026-06-20T09:01:13.388Z)

wasb_infer.py sibling exists: True

45. assistant (2026-06-20T09:01:26.832Z)

I have completed my analysis. Let me compile my findings.

Summary of analysis

No-WSL / no-conda-activate / no-mnt guarantee: CLEAN. `native_wasb_runner.py` imports only `DetectorRunner` (the ABC) and `TrackNetRunner` (for static windowing helpers). It does NOT import or inherit `WslCommandMixin`, and does NOT import `to_wsl_mnt_path` or `wsl_tilde_quote`. The argv is pure list-form (`[python_bin, "wasb_infer.py", "--video", ...]`) with no shell, no `wsl`, no `conda activate`, no `/mnt` translation. `test_is_not_a_wsl_runner` locks this. No leak found.

Parity (cuda default): PRESERVED. `_build_cfg(device="cuda")` ? `gpus="[0]"` ? byte-identical Hydra overrides to the old hardcoded `runner.device=cuda` / `runner.gpus=[0]`. The native runner passes `--device cuda` explicitly, which equals `wasb_infer.main()`'s default. The device-keyed manifest uses `.get("device", "cuda")` so pre-existing caches stay compatible.

Cross-platform (runs on Linux, tested on Windows): correct for production. On real Linux paths, `os.path.abspath`, `expanduser`, `os.path.join(repo,"src")` all behave correctly. The `norm = video.replace("\\","/")` only drives the basename/key (so Windows test paths yield a clean key on Linux); the production `--video` value uses `os.path.abspath(video)` on a real Linux path, which is fine. I confirmed `config.get("indexing")` returns a dict through the worker, and `NativeWasbConfig.from_indexing_cfg` reads it correctly.

Verification: full suite 991 passed; native+worker tests 43 passed; ruff clean; mypy clean on the new files.

The findings below are minor/nit ? no blockers or majors. The change is sound for its acceptance gate (which is owner-side byte-parity on the GPU box).

46. user (2026-06-20T09:01:48.989Z)

Structured output provided successfully

47. assistant (2026-06-20T09:02:05.378Z)

I have completed the no-WSL / cross-platform skeptic review of the M3.5 native WASB runner change.

Verdict

No blockers or majors. The change is sound against its stated contracts. Findings are all nits.

No-WSL guarantee: CLEAN (the core concern)
`backend/pipeline/detectors/native_wasb_runner.py` carries zero WSL plumbing:
- Imports only `DetectorRunner` (ABC) and `TrackNetRunner` (static windowing helpers) ? NOT `WslCommandMixin`, NOT `to_wsl_mnt_path`, NOT `wsl_tilde_quote`.
- argv is pure list-form: `[python_bin, "wasb_infer.py", "--video", abspath, ...]` ? no shell, no `wsl`, no `conda activate`, no `/mnt` translation. The `wasb` env python is invoked by path (`python_bin`).
- `test_is_not_a_wsl_runner` locks `not isinstance(r, WslCommandMixin)` and `not hasattr(r, "_wsl_bash")`.

Parity (cuda default): PRESERVED (the acceptance gate)
- `_build_cfg(device="cuda")` ? `gpus="[0]"` ? byte-identical Hydra overrides to the old hardcoded `runner.device=cuda` / `runner.gpus=[0]`.
- Native passes `--device cuda` explicitly = `wasb_infer.main()`'s default.
- Device-keyed manifest uses `.get("device", "cuda")`, so pre-device caches stay CUDA-compatible.
- Same CSV name `{stem}__{vid12}_wasb.csv`, same `default_cache_dir` (next to `--out`) ? downstream + the owner's byte-parity diff line up.

Cross-platform: correct for production Linux
Verified `os.path.abspath`/`expanduser`/`os.path.join(repo,"src")` all behave on real Linux paths; `norm = video.replace("\\","/")` only feeds the basename/key so Windows test paths yield clean keys on Linux; `config.get("indexing")` correctly returns a dict through the worker train path into `NativeWasbConfig.from_indexing_cfg`.

Verification performed (pre-LGTM gate)
- `python run_all_tests.py` ? 991 passed.
- `tests/test_native_wasb_runner.py` + `tests/test_worker.py` ? 43 passed.
- `ruff check` on all 7 changed/new files ? clean.
- `mypy` on `native_wasb_runner.py` + `worker/__main__.py` ? clean.

The three nits (raw-vs-normalized `--video` path symmetry at `native_wasb_runner.py:163`; `<out>_wasbcache` not reclaimed on success ? pre-existing parity behavior, not a regression; `scratch_dir` env-only with no config knob) are detailed in the structured output. None affect the GPU-box byte-parity acceptance gate.

Files reviewed (all absolute): `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py`, `...\wasb_infer.py`, `...\wasb_runner.py`, `...\base.py`, `...\tracknet_runner.py`, `...\segmenters\trajectory_hybrid.py`, `...\wasb_hybrid.py`, `...\fusion_hybrid.py`, `...\config\models.py`, `...\worker\__main__.py`, `...\tests\test_native_wasb_runner.py`, `...\tests\test_worker.py`.